UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola d'Enginyeria de Telecomunicació
i Aeroespacial de Castelldefels

TRABAJO FINAL DE GRADO

TÍTULO: Desarrollo de una plataforma de juegos y escenarios en realidad aumentada para drones FPV

TITULACIÓN: Grado en Ingeniería de Sistemas Aeroespaciales

AUTOR: Adrià Pascual I Barez

DIRECTOR: Miguel Valero García

FECHA DEPÓSITO: 23/10/2026

TÍTULO: Desarrollo de una plataforma de juegos y escenarios en realidad aumentada para drones FPV

TITULACIÓN: Grado en Ingeniería de Sistemas Aeroespaciales

AUTOR: Adrià Pascual I Barez

DIRECTOR: Miguel Valero García

FECHA DEPÓSITO: 23/10/2026

Resumen

Este proyecto está integrado en el Drone Engineering Ecosystem (DEE) y tiene como objetivo ampliar sus capacidades mediante la realidad aumentada. El objetivo principal ha sido desarrollar una solución que permita integrar de forma realista objetos virtuales en entornos reales y aplicarla a una aplicación para drones de visión en primera persona (FPV).

El software se ha diseñado tanto para fines lúdicos como prácticos. Incluye varios juegos que permiten a los usuarios interactuar con objetos virtuales mientras pilotan un dron. También incorpora un editor de escenarios que permite crear entornos en cualquier parte del mundo mediante una amplia variedad de objetos y elementos virtuales. Esto permite diseñar rutas, circuitos de obstáculos y diferentes escenarios, demostrando el potencial de la realidad aumentada en las aplicaciones con drones.

También se han desarrollado dos aplicaciones web para facilitar la interacción con la plataforma. Estas permiten acceder al sistema desde un dispositivo con conexión a internet, controlar el dron, enviar instrucciones de vuelo y visualizar la aplicación de realidad aumentada mediante una transmisión de vídeo. Una aplicación web está destinada a los pilotos que necesitan controlar el dron, mientras que la otra está diseñada para los observadores que únicamente desean visualizar la aplicación sin utilizar la interfaz de control.

La aplicación de realidad aumentada se ha desarrollado utilizando Unity, que proporciona las herramientas necesarias para crear entornos virtuales, implementar juegos y mostrar objetos virtuales integrados en escenarios reales. Las aplicaciones web se han creado con Flask, una biblioteca de Python que gestiona las peticiones web y la comunicación entre los componentes del sistema. Se utiliza MQTT para intercambiar mensajes y transmitir instrucciones, mientras que WebRTC permite realizar transmisiones de vídeo con baja latencia.

Finalmente, se ha utilizado el simulador Software-in-the-Loop (SITL) de Mission Planner para probar la aplicación sin necesidad de utilizar siempre el dron físico. Esto ha permitido verificar el funcionamiento del sistema y probar las instrucciones de vuelo en un entorno controlado. En conjunto, el proyecto proporciona una base para combinar la realidad aumentada, el control de drones y los escenarios interactivos dentro del Drone Engineering Ecosystem, ofreciendo posibilidades de mejora y ampliación mediante la incorporación de nuevas funcionalidades.

TITLE: Development of a Platform for Augmented Reality Games and Scenarios for FPV Drones

DEGREE: Bachelor's Degree in Aerospace Systems Engineering

AUTHOR: Adrià Pascual I Barez

DIRECTOR: Miguel Valero García

SUBMISSION DATE: 23/10/2026

Abstract

This project is integrated into the Drone Engineering Ecosystem (DEE) and aims to expand its capabilities through augmented reality. The main objective has been to develop a solution that realistically integrates virtual objects into real-world environments and apply it to an application for first-person view (FPV) drones.

The software has been designed for both recreational and practical purposes. It includes several games that allow users to interact with virtual objects while controlling a drone. It also features a scenario editor that enables users to create environments anywhere in the world using a wide variety of virtual objects and elements. This allows users to design routes, obstacle courses, and different scenarios, demonstrating the potential of augmented reality in drone applications.

Two web applications have also been developed to facilitate interaction with the platform. They allow users to access the system from an internet-connected device, control the drone, send flight commands, and view the augmented reality application through a video stream. One web application is intended for pilots who need to control the drone, while the other is designed for observers who only want to watch the application without using the control interface.

The augmented reality application has been developed using Unity, which provides the tools needed to create virtual environments, implement games, and display virtual objects within real-world scenarios. The web applications have been built using Flask, a Python framework that manages web requests and communication between system components. MQTT is used to exchange messages and transmit commands, while WebRTC enables video streaming with low latency.

Finally, Mission Planner's Software-in-the-Loop (SITL) simulator has been used to test the application without always requiring the physical drone. This has made it possible to verify system functionality and test flight commands in a controlled environment. Overall, the project provides a foundation for combining augmented reality, drone control, and interactive scenarios within the Drone Engineering Ecosystem, with opportunities for future improvements and additional features.

A mis padres por todo su apoyo
durante todo el proceso.
A mi tutor por toda la paciencia y dedicación
durante todo el desarrollo del proyecto.

Tabla de contenidos

INTRODUCCIÓN **8**

1 CAPÍTULO 1: CONTEXTO DEL TRABAJO **10**

1.1 Dron **10**

1.1.1 Frame 10

1.1.2 Autopiloto 10

1.1.2 Emisora y receptor de radio 10

1.1.3 Radios de telemetría 11

1.1.4 Cámara FPV 11

1.1.5 Motores 11

1.1.6 Batería 11

1.1.7 GPS 11

1.1.8 Raspberry pi 11

1.2 Dron Engineering Ecosystem **12**

1.2.1 Aplicaciones On Board 12

1.2.2 Aplicaciones Front End 12

1.2.3 Aplicaciones Back End 12

1.2.4 Mi contribución al DEE 13

2 CAPÍTULO 2: OBJETIVOS Y ESTRUCTURA DEL TRABAJO **15**

2.1 Objetivos **15**

2.2 Lista de tareas **15**

2.3 Evolución del trabajo **17**

3 CAPÍTULO 3: HERRAMIENTAS Y TECNOLOGÍAS USADAS **18**

3.1 C# **18**

3.2 Mission Planner **18**

3.3 csDronLink **18**

3.4 MAVLink **19**

3.4 Unity **20**

3.5 NAT **25**

3.6 WebRTC **26**

3.7 HTTP **27**

3.8 MQTT **27**

4 CAPÍTULO 4: EXPLICACIÓN FUNCIONAL DE LA APLICACIÓN **28**

4.1 Menú Inicio **28**

4.2 Menú selección de misión **28**

4.3 Editor de misiones 30
4.4 Menú configuración de misión 36
4.5 Juegos 37
4.5.1 Juego creado en el editor 37
4.5.2 Laberintos 38
4.5.3 Disparar al blanco 39
4.6 WebApp 43
5 CAPÍTULO 5: ARQUITECTURA SOFTWARE 46
5.1 Menú inicio 46
5.2 Menú selección de misión 50
5.3 Editor de misiones 50
5.4 Menú configuración 52
5.5 Juegos 52
5.5.1 Juego creado en el editor 52
5.5.2 Disparar al blanco 58
5.5.3 Laberintos 61
5.6 WebApp 62
6 CAPÍTULO 6: PRUEBAS 67
6.1 Pruebas de instalación 67
6.2 Pruebas en simulación 67
6.3 Pruebas en producción 68
7 CAPÍTULO 7: INTEGRACIÓN AL DRON ENGINEERING ECOSYSTEM 69
8 CAPÍTULO 8: SOSTENIBILIDAD DEL PROYECTO 70
8.1 ¿Quién se beneficia del proyecto? 70
8.2 Escalabilidad del proyecto 70
8.3 Coste del proyecto 70
9 CAPÍTULO 9: CONCLUSIONES 71
9.1 Objetivos alcanzados 71
9.2 Líneas de desarrollo abiertas 71
9.3 Valoración personal 72
REFERENCIAS 73

Índice de figuras

Figura 0.1: Imagen del *dronlab* del campus UPC Baix Llobregat (EETAC)..... 8
Figura 1.1: Imagen del dron usado en el proyecto volando en el *dronlab*..... 10
Figura 1.2: Esquema del *Dron Engineering Ecosystem*..... 13
Figura 3.1: Interfaz de *Unity.*..... 20
Figura 3.2:Ventana *scene* de *Unity.*..... 20
Figura 3.3 Panel *Hierarchy.*..... 21
Figura 3.4 Panel *Project*..... 22
Figura 3.5: Carpeta *Plugins* dentro del panel *Project*..... 22
Figura 3.6 Carpetas dentro dentro de *Assets*: a) *Resources b) Prefabs c) Materials* 23
Figura 3.7: Carpeta *Scenes* dentro del panel *Project*..... 23
Figura 3.8: Carpeta *Shaders* dentro del panel *Project.*..... 24
Figura 3.9: Ventana *Inspector* del elemento *Dron.* 25
Figura 3.10: Ventana *Game.*..... 25
Figura 4.1: Menú de inicio..... 28
Figura 4.2: Menú de selección de Misión..... 29
Figura 4.3 Botones de edición y eliminar misión..... 29
Figura 4.4: En la imagen a) se observa la ventana de eliminar misión y en la b) se visualiza el atajo para entrar al menú de edición o para borrar el escenario. 29
Figura 4.5:Menú de edición de misiones. 30
Figura 4.6 Disminución y aumento del tamaño del panel de herramientas..... 30
Figura 4.7: Botón “Usar mi ubicación” para indicar posición geográfica del usuario..... 31
Figura 4.8: Filtros disponibles para el mapa del mundo en el menú de edición..... 31
Figura 4.9: Previsualizado de objetos con los diferentes parámetros disponibles para modificar. 31
Figura 4.10: Imágenes de todos los elementos disponibles en el menú de edición..... 32
Figura 4.11 Emoticono del objeto *Aro*..... 32
Figura 4.12:Diferentes ejemplos del color de los aros que aparece en los emoticonos..... 33
Figura 4.13: Ventana de copiar, borrar o mover emoticono. 33
Figura 4.14: Líneas de ayuda para colocar objetos alineados. 34
Figura 4.15: Desplazamiento y rotación de una línea de objetos. 34
Figura 4.16: Copias de una línea de elementos. 34
Figura 4.17: Imágenes en las que se observa el previsualizado de misiones..... 35
Figura 4.18: Selector de coordenadas del cursor. 35
Figura 4.19: Casilla para elegir el nombre de la misión..... 36

Figura 4.20: Interfaz del menú de configuración de misión. 36
Figura 4.21: Configuración seleccionada para jugar en modo producción. 37
Figura 4.22: Imagen de la escena del juego “Prueba 3” con el HUD de vuelo activado. 38
Figura 4.23: Interfaz del menú de pausa. 38
Figura 4.24: Imagen de la escena “Laberinto 2”. 39
Figura 4.25: Flechas orientativas del juego “Laberinto 2”. 39
Figura 4.26: Interfaz del menú de configuración del juego “Disparar al blanco”. 40
Figura 4.27: HUD de vuelo del juego “Disparar al blanco”. 40
Figura 4.28: Imagen del juego “Disparar al blanco” con dos enemigos designados. 41
Figura 4.29: Designador de los enemigos en rojo. 41
Figura 4.30: Proyectil *LASER* disparado por el jugador. 41
Figura 4.31: Designador del *MISSILE*. 42
Figura 4.32: Proyectil *MISSILE* disparado al objetivo. 42
Figura 4.33: Pantalla de inicio de la *WebApp*. 43
Figura 4.34: Selección del puerto COM al que esté conectada la antena de telemetría. 43
Figura 4.35: Interfaz de la *WebApp* con el juego ejecutándose desde *Unity*. 44
Figura 4.36: Menú desplegable de la *WebApp*. 44
Figura 4.37: Interfaz de la *WebApp* en el juego “Disparar al blanco”. 45
Figura 5.1: *Hierarchy* del escenario *Inicio*. 46
Figura 5.2 *Inspector* del objeto *Inicio*. 46
Figura 5.3: *Hierarchy* del escenario *Inicio* en ejecución. 49
Figura 5.4: *Hierarchy* del escenario *Menú*. 50
Figura 5.5: *Hierarchy* del escenario *Menú* en el editor de misiones en ejecución. 51
Figura 5.6: *Hierarchy* del escenario *Menú* en el editor de misiones y con el previsualizado de misión abierto. 51
Figura 5.7: *Hierarchy* del escenario *Configuración*. 52
Figura 5.8: *Hierarchy* de los juegos creados desde el editor en ejecución. 53
Figura 5.9: Parámetros del elemento *Camera*. 57
Figura 5.10: Comparación entre ventana *scene* y ventana *Game* cuando el juego está en ejecución. 57
Figura 5.11: *Hierarchy* del juego *Disparar al blanco* cuando el juego está en ejecución. 58
Figura 5.12: Comparación sobre cómo se observa el elemento *dron* en la ventana s*cene* y como se visualiza en el menú de configuración del juego. 59
Figura 5.13: *Hierarchy* de la escena *shooter* correspondiente al juego *Disparar al blanco*. 59
Figura 5.14: *Hierarchy* de la escena *Laberinto 2*. 61

Índice de tablas

Tabla 2.1: Diagrama de Gantt de la planificación temporal del trabajo. 17
Tabla 5.1 Función *Start()* del script *Inicio.cs.* 47
Tabla 5.2: Función *GetMaterial()* del script *Inicio.cs.* 47
Tabla 5.3: Función *Tick()* del script *Inicio.cs*. 48
Tabla 5.4: Función *Start()* del script *Shadertimekeeper.cs*. 49
Tabla 5.5: Función *Bootstrap()* del script *DronMqttBridge.cs.* 53
Tabla 5.6: Función *Awake()* del script *DronMqttBridge.cs.* 54
Tabla 5.7 Función *Start()* del script *DronMqttridge.cs*..... 54
Tabla 5.8: Extractos de la función *ConnectToBrokerAsync()* del script *DronMqttBridge.cs.* 55
Tabla 5.9: Función *Update()* del script *DronControl.cs.* 56
Tabla 5.10: Función *GetTelemetrySnapShot()* del script *DronMqttBridge.cs.* 56
Tabla 5.11: Ejemplo de diferentes rutas HTTP con funciones asociadas. 62
Tabla 5.12: Creación del cliente MQTT para conexión con servidor *HiveMQ*. 63
Tabla 5.13: Función *run_in_background()* del script *webrtc_server.py*. 65
Tabla 5.14: *Función _start_websocket_server* del script *webrtc_server.py*. 65

Glosario

DEE	Drone Engineering Ecosystem
AR	Augmented Reality
FPV	First Person View
UAV	Unmanned Aerial Vehicle
SITL	Software In The Loop
MAVLink	Micro Air Vehicle Link
MQTT	Message Queuing Telemetry Transport
GPS	Global Positioning System
JSON	JavaScript Object Notation
HUD	Head-Up Display
DLL	Dynamic-Link Library
UI	User Interface
IDE	Integrated Development Environment
HTTP	HyperText Transfer Protocol
RTC	Real-Time Communication
TCP	Transmission Control Protocol
SDP	Session Description Protocol
ICE	Interactive Connectivity Establishment
UAV	Unmanned Aerial Vehicle

INTRODUCCIÓN

Los drones son dispositivos aéreos muy usados hoy en día para múltiples tareas en el ámbito agrícola, cinematográfico, militar o recreacional. Son muy atractivos debido a la facilidad de su uso y al bajo coste que estos tienen.

En este trabajo se ha desarrollado una aplicación de realidad aumentada para drones FPV (First Person View). Este tipo de UAV se caracteriza por disponer de una cámara a bordo que permite al piloto visualizar la perspectiva del dron durante el vuelo.

El programa permite al usuario visualizar objetos virtuales dentro de un escenario real. Es una herramienta útil tanto para fines lúdicos como prácticos. La plataforma incluye un editor que permite al usuario crear sus propios circuitos y misiones. Además, la aplicación también contiene diferentes modos de juego basados en disparar a enemigos virtuales o en escapar de laberintos.

Se ha desarrollado este proyecto con la finalidad de que sea una herramienta que pueda ser utilizada por los visitantes al *dronlab* (ver figura 1) del campus del Baix Llobregat (UPC). Este es un recinto protegido con redes de seguridad que cumple con la normativa vigente sobre el vuelo de vehículos aéreos no tripulados. En este espacio se pueden volar todo tipo de drones para realizar pruebas o diferentes actividades.



Figura 0.1: Imagen del *dronlab* del campus UPC Baix Llobregat (EETAC).

Gracias a esta aplicación los visitantes podrán disfrutar de una experiencia de realidad aumentada con drones, con la posibilidad de controlar el dron ellos mismos a través de la *WebApp* que incluye el programa.

La motivación personal para realizar este proyecto ha sido el interés en ampliar mi conocimiento y mejorar mi comprensión sobre el funcionamiento de los drones. En este proyecto he podido profundizar en los mecanismos que usan estos para la transmisión y recepción de información y en las diferentes herramientas de programación que son usadas para el diseño de juegos 3D o páginas web.

La memoria está estructurada en diferentes apartados en los que se explican los diferentes aspectos de la aplicación. En el primero se hace una descripción sobre los diferentes componentes que incluyen los drones que usan este programa y se hace una explicación sobre que es el *Dron Engineering Ecosystem* (DEE). En el segundo se hace una enumeración de todos los objetivos y tareas que se habían planteado para el desarrollo de la aplicación. También se incluye una planificación temporal donde se detalla la evolución del trabajo a lo largo del tiempo.

En el siguiente se hace una descripción sobre el funcionamiento de todas las herramientas software y hardware que han sido necesarias para el desarrollo de todo el proyecto.

En el quinto ya se enseña la aplicación acabada y se hace una descripción funcional donde se detallan todas las características que tiene el programa. Posteriormente a este, se hace una explicación sobre toda la arquitectura software de la aplicación, profundizando en todos los aspectos de esta. En el séptimo se realiza un análisis sobre todas las conexiones de red que son usadas durante la ejecución del juego para mostrar cómo se comunican los diferentes dispositivos en tiempo real. El siguiente consiste en una enumeración de todas las pruebas que debe realizar el usuario antes de empezar a utilizar la aplicación con el dron real, para de esta forma garantizar que todo funcione correctamente. Finalmente, en los tres últimos apartados se hace un estudio sobre el impacto medioambiental que puede haber tenido el desarrollo de esta aplicación, se realiza una declaración sobre el uso que se ha hecho de la IA como herramienta de ayuda, y finalmente se explican las conclusiones del proyecto, detallando los objetivos alcanzados, la valoración personal final de todo el trabajo y la propuesta de posibles vías de desarrollo futuras para los estudiantes que quieran mejorar esta herramienta.

CAPÍTULO 1: CONTEXTO DEL TRABAJO

En este apartado se va a mostrar el Hardware que se ha utilizado para la realización de este trabajo y se explicará cuál es el ecosistema en el que se van a integrar todas las aplicaciones creadas en este proyecto.

1.1 Dron

El dron está compuesto por diferentes partes que hacen posible que sea capaz de realizar todas las tareas necesarias para el desarrollo de este proyecto (ver figura1.1).



Figura 1.1: Imagen del dron usado en el proyecto volando en el *dronlab*.

1.1.1 Frame

El dron que se ha usado está construido a partir del *frame* HEXSOON EDU450. Es de fibra de carbono y tiene una gran resistencia contra impactos.

1.1.2 Autopiloto

El modelo usado es el *Pixhawk Orange Cube*. El autopiloto da la capacidad al dron de poder realizar diferentes acciones de forma autónoma sin la intervención del piloto, como puede ser despegar, aterrizar, volver al punto de inicio, etc. Incluso se le pueden programar diferentes planes de vuelo para que el dispositivo los haga de forma completamente autónoma sin intervención humana. Esto es posible gracias al módulo GPS que lleva incorporado el dron.

1.1.2 Emisora y receptor de radio

La emisora es el modelo *FrySky taranis* y el receptor de radio usado es el *FrSky X8R ACCST*. Las instrucciones que son recibidas por el dron son procesadas por el autopiloto. Este es el encargado de llevar a cabo las acciones enviadas por la emisora.

1.1.3 Radios de telemetría

Es un tipo de antena que puede ser conectada a un puerto USB. En el ecosistema hay una en la estación de tierra y otra conectada directamente al dron. De esta forma se pueden enviar datos bidireccionalmente.

1.1.4 Cámara FPV

La cámara usada es una cámara FPV. Esta captura la imagen desde la perspectiva del dron. Es la que proporciona el vídeo necesario para hacer la aplicación de realidad aumentada. La imagen se envía a la estación de tierra a través de una antena que a la vez está conectada a un VTX (*video transmitter*) que procesa la imagen digital de la cámara y la convierte en una señal que puede ser enviada en forma de ondas electromagnéticas. Es importante tener en cuenta que este tipo de sensores capturan el video con una cierta distorsión llamada ojo de pez. Con tal de que el efecto AR sea lo más realista posible, es importante corregir esta distorsión.

1.1.5 Motores

Los motores son unos motores eléctricos sin escobillas controlados por los llamados ESC (*Electronic Speed Controller*). Estos transforman la corriente continua de la batería en corriente trifásica para conseguir crear un campo magnético giratorio a través de las bobinas y lograr que el rotor gire.

1.1.6 Batería

Las baterías *lipo* son las más usadas para los drones. Estas tienen una alta densidad energética, lo cual es idóneo ya que se busca minimizar el peso al mínimo.

1.1.7 GPS

El dron también incluye un módulo GPS que permite al *autopilot* saber en todo momento las coordenadas que el UAV ocupa en el espacio. Es importante saber estos datos de cara a hacer misiones autónomas. También es importante para la aplicación de este trabajo, ya que sin las coordenadas reales del dron no sería posible crear correctamente el efecto de realidad aumentada.

1.1.8 Raspberry pi

Es una placa electrónica que actúa como un ordenador al que se le puede instalar un sistema operativo como Linux. Es útil porque permite controlar y monitorear diferentes dispositivos del dron a través de programas que pueden ser modificados directamente por el usuario.

1.2 Dron Engineering Ecosystem

El DEE es un ecosistema modular que incluye múltiples aplicaciones software para drones. Ha sido desarrollado a partir de la contribución de diferentes alumnos y docentes. En este hay todo tipo de programas que ofrecen múltiples funcionalidades para los UAVs, como reconocimiento de imágenes, control del dron vía *WebApp*, transmisión de vídeo a múltiples usuarios, etc. En la figura 1.2 se puede observar un esquema de la organización del DEE. Las aplicaciones se pueden dividir en tres tipos:

1.2.1 Aplicaciones On Board

Estas son módulos que se ejecutan en la *Raspberry PI* del dron. Controlan diferentes dispositivos como el autopiloto, la cámara, *LEDs*, servos, etc. Están programados en *Python*.

1.2.2 Aplicaciones Front End

Estas son las aplicaciones con las que interactúa directamente el usuario. Suelen tener una interfaz gráfica y sirven para enviar instrucciones al dron, procesar datos o imágenes enviados por este, diseñar *flight plans*, etc. Entre las *Front End* hay aplicaciones de escritorio hechas con *C#* o *Python*, *apps* nativas para móviles o tabletas programadas en *Java*, *Kotlin* o *Flutter*, y *WebApps* a las que el usuario puede acceder por internet y que usan tecnologías como *Flask* o *VUE*.

Por lo que respecta a aplicaciones de realidad aumentada dentro del DEE, previamente a este trabajo se había realizado otro proyecto hecho por el alumno Arnau Domènech. Este también se había realizado en *Unity* y sirvió como referencia para este TFG, aunque la aplicación de este proyecto haya sido desarrollada desde cero sin utilizar recursos de desarrollos anteriores.

1.2.3 Aplicaciones Back End

Estos son programas destinados al procesamiento y almacenamiento de datos. Todas estas aplicaciones pueden ser usadas con el dron real o con el SITL de *Mission Planner*, el cual simula el vuelo de un UAV dentro de un servidor y permite interactuar con este de la misma forma que con el dron real, ya que utiliza los mismos protocolos de comunicación.

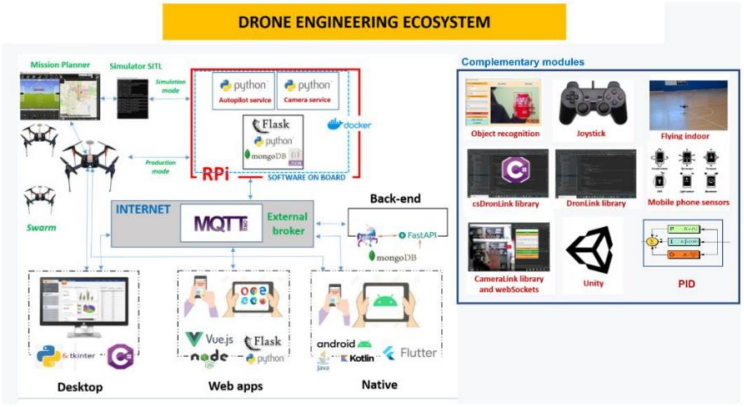


Figura 1.2: Esquema del *Dron Engineering Ecosystem*.

1.2.4 Mi contribución al DEE

Mi trabajo consta de dos aplicaciones que funcionan de forma sinérgica. La primera es una *app* de escritorio hecha en *Unity* que permite jugar a juegos que usan realidad aumentada.

Para conseguir un efecto realista y que el usuario tenga la sensación de que los objetos digitales están realmente en el escenario real se tienen que seguir una serie de pasos. El primero es conseguir que un objeto, como puede ser un cubo, en un escenario 3D de *Unity* se mueva de la misma forma que el dron real. Es decir, que, si el dron avanza 5 m a la derecha, que el dron virtual también realice ese desplazamiento. Esto se consigue obteniendo las coordenadas GPS que el UAV real envía a través de su antena de telemetría. Al tener otra antena dentro del rango de cobertura de la otra conectada al ordenador en el que se está ejecutando el programa AR, el objeto que actúa como dron virtual en el escenario de *Unity* es capaz de obtener esos datos y hacer el mismo movimiento. También es necesario obtener información sobre los giros que hace el dron sobre sus respectivos ejes, los llamados *roll*, *pitch* y *yaw*. El dron obtiene estos datos de actitud mediante IMUs (*Inertial Measurementents Units*) que lleva incorporados el autopiloto *OrangeCube*, y los envía a la estación de tierra a través de la antena de telemetría. Finalmente, el último dato que envía el cuadricóptero a *Unity* por este canal de comunicación es la altitud. Esta información el cuadricóptero la consigue gracias a los altímetros que tiene instalados. Con todos estos datos, el objeto de *Unity* se moverá de la misma forma que el UAV real. En este punto, el siguiente paso es conseguir que la imagen de vídeo del dron real se vea en el juego. Para esto, habría que añadirle el objeto cámara de *Unity* al elemento que simula el vuelo real del cuadricóptero, para de esta forma conseguir una perspectiva desde el dron virtual del escenario de Unity parecida a la de la cámara del dron físico. Es importante que tanto la cámara real como la virtual tengan el mismo campo de visión. Una vez hecho esto, el siguiente paso sería obtener el vídeo de la cámara del dron. Este es enviado por un canal de comunicación separado al de telemetría, pero que funciona también con los mismos principios.

Con el vídeo real ya en Unity, solo habría que renderizar este encima de un objeto con forma de pantalla que estuviese siempre situado delante de la cámara virtual y que ocupase todo el campo de visión de esta, para que así pueda captar el vídeo real y plasmarlo en la pantalla final del juego. En este punto, la aplicación ya está preparada para que se añadan los objetos virtuales alrededor del escenario virtual de Unity que se quieran ver en realidad aumentada. Si alguno de estos elementos entra en el campo de visión de la cámara del dron virtual, estos objetos serán visualizados por el usuario desde la pantalla del juego superpuestos sobre el vídeo del escenario real que transmite el UAV físico.

La aplicación incluye menús de selección, de configuración, y un editor de misiones en el que el usuario puede crear sus propios escenarios, pudiendo hacer gran variedad de circuitos y distintos juegos.

Además de las misiones creadas por el propio jugador, la aplicación trae consigo dos modalidades de juego: los “laberintos” y “disparar al blanco”. En el primer caso el objetivo principal es que el usuario consiga escapar de un laberinto virtual, las paredes del cual no puede atravesar. En cuanto al juego de disparar, este consiste en lanzar proyectiles a diferentes enemigos que se mueven de forma aleatoria alrededor de un recinto cerrado. Estos pueden dispararte y eliminarte. A medida que vas derribando a los enemigos van apareciendo nuevos y aumentando la dificultad de estos.

La segunda aplicación es una *Webapp.* Esta tiene la finalidad de que el usuario pueda controlar el dron mediante su móvil o cualquier dispositivo con acceso a internet. También incluye controles para poder jugar a los diferentes juegos, como un botón para disparar, otro para cambiar de blanco, etc. Además, la aplicación de *Unity* envía los *frames* de vídeo del propio juego a la *Webapp* vía *WebRTC*, para que el usuario pueda observar en la pantalla del dispositivo con el que esté conectado, el juego que se está ejecutando en el ordenador. La plataforma también incluye otra *WebApp* destinada a observadores. En esta solo se ve la imagen de la aplicación de *Unity* y no ofrece la posibilidad de enviar órdenes al dron ni manejar los controles de ningún juego.

CAPÍTULO 2: OBJETIVOS Y ESTRUCTURA DEL TRABAJO

En este apartado se listarán todos los objetivos que se habían propuesto previamente a la realización del proyecto, se enumerarán las diferentes tareas necesarias para completar todas las metas y se representará la evolución del trabajo a lo largo del tiempo en un diagrama de Gantt.

2.1 Objetivos

- Crear una aplicación en *Unity* con una interfaz gráfica de usuario intuitiva y un diseño moderno y tematizado.
- Desarrollar un menú de edición de misiones en el que el usuario pueda diseñar sus propios escenarios en cualquier sitio del mundo real, con una amplia variedad de objetos y colores.
- Dar la opción al usuario de poder jugar conectado al dron real o al SITL de *Mission Planner*. La fuente de vídeo para el fondo del juego también debe poder seleccionarse libremente entre el conjunto de dispositivos de cámara que estén conectados al ordenador en el que se esté ejecutando la aplicación.
- Simular el mundo real en *Unity*, de manera que cada coordenada geográfica que envíe el dron físico corresponda a un punto determinado del escenario plano virtual.
- Ofrecer una experiencia de realidad aumentada realista, con una jugabilidad fluida y agradable.
- Crear distintos juegos para que el usuario pueda probar las diferentes posibilidades que ofrece la realidad aumentada.
- Crear una *WebApp* desde la que el usuario pueda controlar el dron real o simulado y pueda visualizar el juego desde la pantalla de su dispositivo.
- Incluir otra *WebApp* que sea utilizada por observadores, en la que solo se pueda visualizar la aplicación de *Unity* sin posibilidad de enviar instrucciones al dron ni interactuar con ningún juego.

2.2 Lista de tareas

Fase 1: Investigación y Documentación

- Exploración del DEE. Entendimiento de la arquitectura *software* implementada en el ecosistema e investigación sobre todas las funcionalidades que ofrecen los diferentes módulos.
- Analizar el proyecto previo de realidad aumentada hecho por Arnau Domènech para entender cómo había dado solución a las diferentes problemáticas que presenta la realidad aumentada.
- Aprendizaje sobre el funcionamiento de *Unity* y planteamiento de diferentes propuestas para resolver los diferentes desafíos asociados a la realidad aumentada.
- Aprendizaje sobre el funcionamiento de los diferentes protocolos de comunicación de red y transporte que utilizan los sistemas operativos de los ordenadores.

Fase 2: Primeras pruebas

- Establecer una conexión entre *Unity* y el SITL de *Mission Planner*.
- Transformar las coordenadas esféricas provenientes del dron físico en coordenadas cartesianas, para que cada punto del mundo real corresponda a una localización específica en el espacio plano 3D de *Unity.*
- Lograr que un objeto dentro de un escenario de *Unity* realice los mismos movimientos que el dron simulado en el SITL.

Fase 3: Realidad aumentada en Unity

- Adjuntar el objeto cámara de *Unity* al elemento que se desplaza igual que el dron real dentro del escenario virtual.
- Agregar un panel rectangular al objeto cámara que esté posicionado siempre delante de esta, que ocupe todo su campo de visión y que sirva como pantalla sobre la cual renderizar el vídeo captado por la cámara del dron real.
- Crear un script que capture el vídeo de la webcam del ordenador y lo renderice sobre el panel rectangular previamente creado.

Fase 4: Interfaz gráfica para el usuario

- Creación y diseño de las interfaces gráficas de usuario correspondientes al menú de inicio, selección, configuración y pausa.
- Agregar un menú de edición de misiones que permita al usuario colocar objetos sobre cualquier localización en el mundo, para que de esta forma el usuario pueda diseñar sus propios circuitos y escenarios.

Fase 5: Creación de los juegos

- Diseño del HUD de vuelo.
- Agregar *geofences* al recinto que abarca el *dronlab*.
- Creación del juego disparar al blanco.

    1. Diseño visual de los enemigos y de los disparos.
    2. Desarrollo del comportamiento y de los patrones de movimiento de los enemigos.
    3. Elaboración del sistema de dificultad del juego.

- Desarrollo de los juegos de tipo laberinto.

    1. Diseño de diferentes distribuciones del espacio con muros en el recinto del *dronlab*.
    2. Agregar *geofences* a las paredes del laberinto.

Fase 6: Creación WebApp

- Crear una interfaz gráfica de usuario que incluya diferentes botones para mandar instrucciones al dron.
- Implementación de un sistema de control del dron dentro de la *WebApp* mediante *joysticks*.
- Creación de diferentes pulsadores para jugar al juego de disparar al blanco.
- Desarrollo de una segunda *WebApp* solo para observadores donde solo se vea la aplicación de *Unity*.

Fase 7: Pruebas

- Realización de pruebas con el dron real en el *dronlab*, fuera del entorno de simulación.
- Corrección de fallos detectados durante las pruebas y mejora de las prestaciones de la aplicación.

Fase 8: Documentación del proyecto

- Redacción de la memoria
- Grabación de vídeos que describan el funcionamiento de la aplicación y expliquen la estructura del código.
- Integración del proyecto en un repositorio público de *GitHub*.

2.3 Evolución del trabajo

En este apartado se mostrará un diagrama de Gantt en el que se puede ver la evolución del proyecto y el espacio de tiempo que ha ocupado cada fase del trabajo (ver tabla 2.1).

	Enero	Febrero	Marzo	Abril	Mayo	Junio	Julio	Agosto	Septiembre
Fase 1									
Fase 2									
Fase 3									
Fase 4									
Fase 5									
Fase 6									
Fase 7									
Fase 8									

Tabla 2.1: Diagrama de Gantt de la planificación temporal del trabajo.

CAPÍTULO 3: HERRAMIENTAS Y TECNOLOGÍAS USADAS

En este apartado se realizará una explicación sobre las diferentes herramientas y tecnologías que se han usado durante el desarrollo del proyecto.

3.1 C#

C# es un lenguaje de programación orientado a objetos. Es el lenguaje nativo de scripting del motor de *Unity*, sobre el que está construida toda la parte de realidad aumentada de este proyecto. En este trabajo, C# se ha utilizado para programar toda la lógica de la aplicación: desde la gestión de escenas y menús (selección de misión, configuración, pausa), hasta el control de los distintos modos de juego.

Además de la interfaz de usuario o la jugabilidad, C# ha sido el lenguaje empleado para toda la capa de comunicación: la conexión *MAVLink* con el piloto automático (tanto en simulación mediante SITL como en producción con hardware real), la publicación de telemetría e instrucciones a través del bróker MQTT, y la captura de vídeo del propio juego para su transmisión en tiempo real mediante *WebRTC* hacia la *WebApp*.

3.2 Mission Planner

Mission Planner es un programa destinado a vehículos que utilicen el *firmware ArduPilot*. Es la herramienta que se ha usado para simular el dron durante el desarrollo de este proyecto. El programa permite crear un servidor llamado SITL en el que se simula el vuelo de un dron real. Este tiene diferentes puertos TCP abiertos con la finalidad de que tanto el propio software de *Mission Planner* como programas externos puedan conectarse al servidor y mandar diferentes instrucciones al UAV utilizando los mismos protocolos de comunicación usados para la transmisión de datos al dron real.

Además de la simulación, Mission Planner ha sido la herramienta empleada para supervisar el estado del dron y enviar diferentes órdenes a este durante las pruebas en simulación y en producción.

3.3 csDronLink

csDronLink es una biblioteca en *C#* desarrollada específicamente para facilitar la comunicación con drones mediante el protocolo *MAVLink*, ya sea con un simulador SITL o con un dron real conectado por radio de telemetría. Proporciona la clase principal “Dron” que encapsula toda la complejidad de bajo nivel del protocolo: apertura y cierre de la conexión, tanto por TCP como por puerto serie, y el envío y recepción de los distintos tipos de mensaje *MAVLink*. También incluye métodos para las operaciones habituales de vuelo: conectar, armar, despegar, aterrizar, volver al punto de origen, cambiar el modo de vuelo, dirigirse a un punto concreto o enviar valores de mando por radio control.

En este proyecto, los métodos incluidos en *csDronLink* son la única vía de comunicación real con el dron, y están integrados dentro del script *DroneMqttBridge.cs*. Este es el responsable tanto de establecer y mantener la conexión (en modo simulación o con el dron real, según la configuración elegida por el usuario) como de recibir de forma continua la telemetría del vuelo, posición, altitud, velocidad, orientación y estado del piloto automático. Además de las operaciones básicas de vuelo, la biblioteca se ha empleado también para la gestión de *geofences*. Estas son barreras ficticias que son enviadas al autopiloto y que permiten delimitar zonas de vuelo concretas para el dron, ya que este se frena antes de llegar a atravesarlas. En este proyecto se han utilizado para delimitar la zona de vuelo del *dronlab* y bloquearle al UAV el paso a través de las paredes de los laberintos.

3.4 MAVLink

MAVLink (*Micro Air Vehicle Link*) es un protocolo de comunicación ligero, de código abierto, diseñado específicamente para el intercambio de información entre vehículos aéreos no tripulados y estaciones de control en tierra. Es el protocolo estándar utilizado por los *firmwares* de autopiloto como *ArduPilot* o PX4, y define un conjunto amplio de tipos de mensaje de telemetría, de estado del sistema y de órdenes de vuelo, codificados en un formato binario compacto pensado para funcionar de forma eficiente incluso sobre enlaces de comunicación de bajo ancho de banda, como es el caso de la comunicación a través de radios de telemetría. Al ser un protocolo independiente del medio de transporte, puede circular indistintamente a través de un puerto serie o de una conexión TCP o UDP. Esto le permite adaptarse tanto a un enlace físico real con el dron como a una conexión con el simulador SITL.

En este proyecto, *MAVLink* es el protocolo que sustenta toda la comunicación de bajo nivel entre la aplicación y el dron, ya sea simulado o real, a través de la biblioteca *csDronLink*. Entre los mensajes empleados destacan *HEARTBEAT* (con el que cada extremo confirma periódicamente que sigue activo, y del que además se extrae el modo de vuelo y el estado de armado), *GLOBAL_POSITION_INT (coordenadas, velocidad y rumbo)* y *ATTITUDE* (ángulo de *pitch*, *yaw* y *roll*). Asimismo, *MAVLink* se ha utilizado también para la gestión de las *geofences*.

3.4 Unity

Unity es un motor de desarrollo de aplicaciones y videojuegos en 2D o 3D. Tiene una interfaz gráfica intuitiva, con diferentes paneles y ventanas que se pueden organizar en diferentes disposiciones según el criterio del usuario. Para el desarrollo de este proyecto, se ha usado la siguiente distribución de interfaz (ver Figura 3.1).

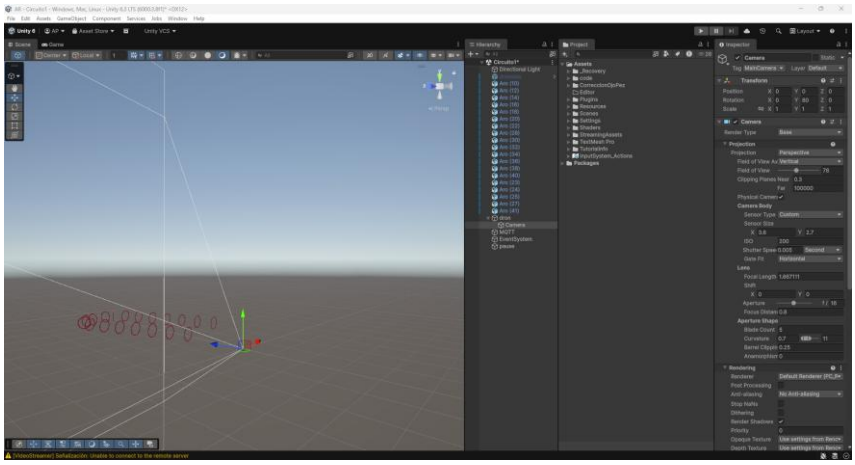


Figura 3.1: Interfaz de *Unity.*

La ventana de más a la izquierda de la imagen X se llama *Scene* (ver figura 3.2). Esta permite al usuario moverse libremente por el escenario 3D, visualizar los diferentes elementos del escenario y colocar de nuevos.

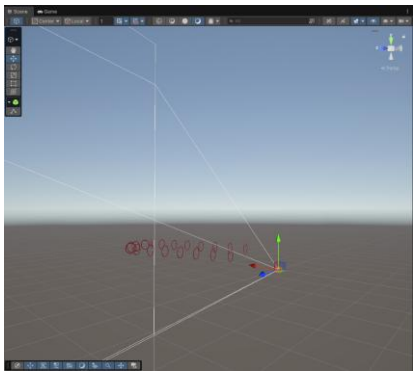


Figura 3.2:Ventana *scene* de *Unity.*

El siguiente panel que se observa es el llamado *Hierarchy* (ver figura 3.3). Sirve para añadir objetos físicos virtuales visibles o no visibles dentro de la escena. Por ejemplo, si se quiere colocar un aro se debe poner el objeto aro en esta ventana para que automáticamente aparezca en el entorno 3D.

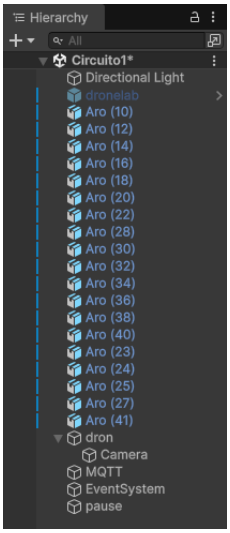


Figura 3.3 Panel *Hierarchy*.

La tercera ventana es la llamada *Project* (ver figura 3.4). En esta aparecen todos los recursos que han sido usados durante el desarrollo de la aplicación.

Dentro de la carpeta *code* aparecen todos los scripts *C#* que han sido usados en la aplicación. Estos para que sean ejecutados durante el juego necesitan ser adjuntados a un objeto que esté dentro del *Hierarchy*. Para ello, directamente desde la ventana de *Project* se arrastra el script hasta colocar el ratón encima del elemento al que se le quiera agregar este archivo. Una vez hecho esto, cuando se ejecute el juego y se cree ese objeto dentro de la escena 3D, el motor de *Unity* se encargará de ejecutar ciertas funciones concretas que viven dentro del script. El objeto no tiene por qué ser visible, *Unity* da la opción de añadir un *empty object* en la escena, el cual no tiene ningún tipo de textura y sirve únicamente para que el script que se le adjunte se ejecute al iniciar el juego. No obstante, el proyecto de *Unity* no deja de ser a la vez una aplicación de *C#*. Por lo tanto, aunque se tenga una clase estática pública dentro de algún script que no esté adjunto a ningún objeto dentro del *Hierarchy*, cualquier código podrá acceder a ella sin limitaciones.

El motor de *Unity* por diseño analiza todos los objetos que existen en el *Hierarchy* antes de iniciar una escena, y si algún elemento tiene un script adjunto, el motor está programado para llamar a diferentes funciones de dentro del código automáticamente, como puede ser la función *Start()* o *Update()*.

El compilador de *Unity* compila todos los ficheros *.cs* que se encuentren dentro de la carpeta *Assets* en un mismo ensamblado, exceptuando los encontrados dentro de la carpeta *Editor* (está vacía en este proyecto). Por lo tanto, todas las clases que existen dentro de los diferentes scripts que hay en *Assets* son accesibles siempre y cuando los modificadores de acceso de estas lo permitan.



Figura 3.4 Panel *Project*.

En la carpeta *plugins* (ver figura 3.5) se encuentran las librerías compiladas *.dll* que son usadas por los diferentes scripts de la carpeta *code*. La primera es csDronLink.dll. También se encuentra *MQTTnet.dll* que es la librería de *C#* que le permite a *Unity* hablar el protocolo MQTT. Las dos restantes son las librerías *SharpDX.dll* y *SharpDX.DirectInput.dll*. Es importante que estén presentes porque *csDronLink* tiene referencias a las librerías *SharpDX.DirectInpu.dll* y *SharpDX.dll*. Sirven para acceder a la API *DirectInput* de Windows, que es la encargada de leer dispositivos de entrada como mandos, *joysticks*, etc. En este proyecto no se usan, pero aun así es importante que existan porque, si faltan estas referencias, el ensamblado de *csDronLink* no se llevará a cabo.

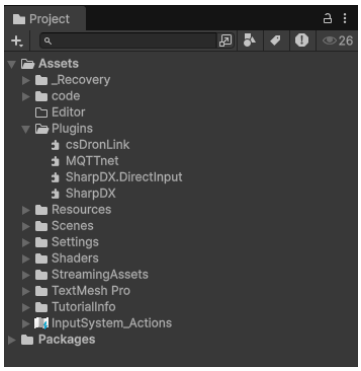


Figura 3.5: Carpeta *Plugins* dentro del panel *Project*.

En la carpeta *Resources* (ver figura 3.6a) se encuentran varios de los modelos 3D y recursos visuales utilizados en el proyecto. En la carpeta *prefabs* (ver figura 3.6b) se encuentran todos los objetos 3D en formato *glTF* del editor de misiones. Se pueden observar en diferentes colores, los aros, las flechas, las banderas, etc.

En la carpeta *Materials* (ver figura 3.6c) aparecen los diferentes materiales que son usados por los diferentes modelos 3D que hay en el proyecto. Un *Material* es una clase que existe en el código del motor de *Unity*. Contiene todos los datos necesarios para pintar una malla 3D.

a) b) c)

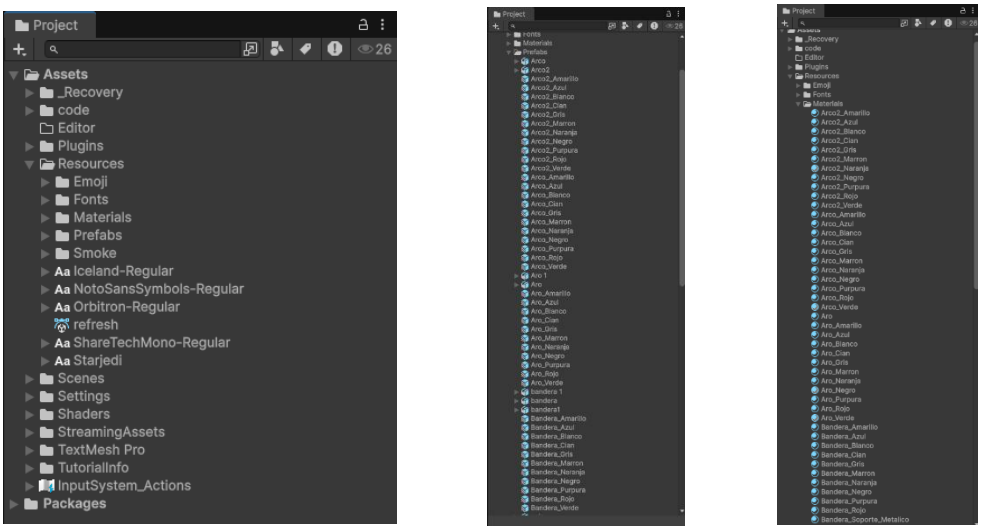


Figura 3.6 Carpetas dentro dentro de *Assets*: a) *Resources b) Prefabs c) Materials*

En la carpeta *scenes* (ver figura 3.7) se encuentran todas las escenas que hay en el proyecto. Una escena es un espacio 3D independiente del resto. Es decir, cada escena tiene sus propios objetos en el *Hierarchy* y sus propios recursos que no son compartidos. Es ideal para separar juegos distintos y diferentes menús. Los escenarios *Inicio*, *Menú* y *Configuración* son espacios en los que se crean las diferentes interfaces de usuario. En *laberinto*, *laberinto2*, *laberinto3* y *shooter*, se crean las escenas donde se van a ejecutar los diferentes juegos del proyecto. La escena *Circuito1* sirve como plantilla para las misiones creadas desde el editor.

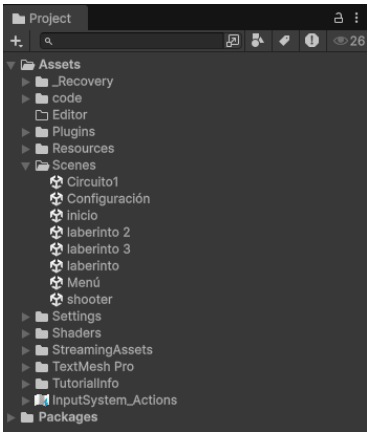


Figura 3.7: Carpeta *Scenes* dentro del panel *Project*.

Por último, se encuentra la carpeta *Shaders* (ver figura 3.8). En esta, entre otras cosas, aparecen algunos de los *shaders* usados en el juego. Son archivos escritos en HLSL (*High-Level Shading Language*) y a diferencia del resto de scripts en *C#* de la aplicación, estos son ejecutados por la GPU. La tarjeta gráfica de un ordenador, a diferencia de la CPU, tiene la capacidad de realizar la misma tarea en paralelo repetidas veces, gracias a la gran cantidad de núcleos que tiene esta pieza de *hardware*. Está diseñada precisamente para tareas que requieran de una gran cantidad de cálculos sencillos, como puede ser pintar los píxeles de la pantalla de un juego. Una CPU también puede ejecutar programas en paralelo porque tiene varios núcleos también, pero muchos menos que la GPU. Sin embargo, los núcleos de la CPU están preparados para realizar tareas mucho más complejas.

Todas las variables de la clase *Material* deben incluir un *shader*. En el *Material* está contenida la información sobre diferentes variables, como pueden ser el color o las texturas, pero la acción de pintar cada píxel del objeto en cuestión con el color correspondiente la realizan los *shaders* usando precisamente variables facilitadas por el *Material*. Por ejemplo, en el caso del fondo animado de los menús, los colores usados por este son información del *Material*, pero el código que se encarga de realizar esa animación se encuentra en el *shader*, porque es el que elige el color que debe tener cada píxel del *Canvas* en un momento concreto. Cuando un *Material* tiene adjunto un *shader*, este ya puede ser adjuntado a un *Prefab*, que, entre otras cosas, contiene la información sobre la malla del modelo 3D del elemento que debe ser pintado. Con todos estos datos, *Unity* ya puede mostrar el objeto 3D en el escenario de juego. Los *shaders* que están detrás de la lógica del fondo animado del menú son el *ARDashboard_Background_39* y el *ARDashboard_FPVBlur*, mientras que el *FisheyeUndistort* se encarga de dibujar los píxeles del elemento que actúa como pantalla en los juegos de *Unity*. En este objeto se renderiza el vídeo real, y el *shader* es el encargado de establecer el color de cada píxel aplicando la corrección de ojo de pez.

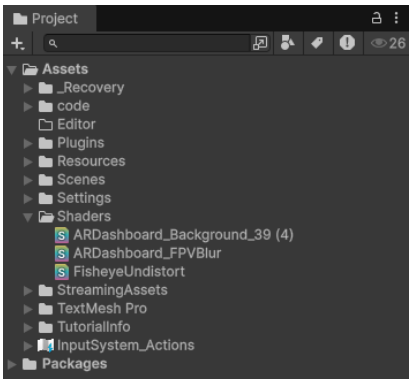


Figura 3.8: Carpeta *Shaders* dentro del panel *Project.*

En la figura 3.9 se observan los datos que aparecen en el *inspector* cuando el objeto *dron* del *Hierarchy* está seleccionado. Los datos de posición son los referentes a este objeto. Se observa que tiene el script *DronControl.cs* adjunto. Incluso se pueden cambiar los valores de algunos parámetros de este código desde esta pestaña. También aparecen diferentes propiedades del elemento, como puede ser la detección de colisiones.

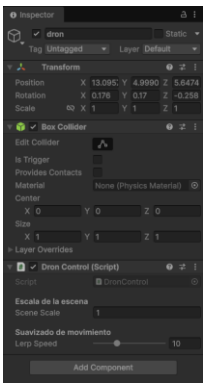


Figura 3.9: Ventana *Inspector* del elemento *Dron.*

En caso de que se quiera ejecutar el juego dentro del entorno de programación de *Unity*, se tiene que pulsar el botón *play* situado en la parte superior derecha de la pantalla. Al seleccionarlo, se abre una nueva ventana llamada *Game* (ver figura 3.10*)* donde se puede observar la aplicación en ejecución. Es muy útil porque permite visualizar como se plasman en el juego final todas las modificaciones realizadas durante el desarrollo.

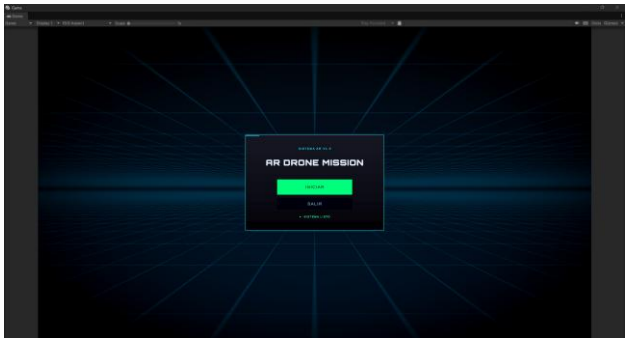


Figura 3.10: Ventana *Game.*

3.5 NAT

Se puede definir como una tabla de traducción de (IP pública: puerto público) a (IP privada: puerto privado). Cuando un dispositivo de una red local quiere conectarse por ejemplo a un servidor web, lo que hace es enviar una petición al *router* a través de una dirección IP y puerto privados. El *router* al recibir esa petición, antes de enviarla a internet, sustituye esa IP y puerto privados por una IP y puerto públicos que son visibles para la red de internet. De esta forma, cuando el servidor web conteste a la petición, enviará la respuesta a esa IP y puerto público. Entonces el *router* recibirá la respuesta, consultará su tabla NAT para saber a qué IP y puerto privados corresponde esa combinación específica de IP y puerto públicos. Hay NATs más restrictivas que otras. Por ejemplo, las de tipo simétrico por cada IP pública distinta asignan un puerto público diferente, aunque el puerto privado sea el mismo.

3.6 WebRTC

WebRTC es un sistema de comunicación bidireccional que permite intercambiar datos como vídeo o audio entre varios dispositivos sin intermediarios. En el caso de este proyecto, esta tecnología está implementada con una configuración de emisor-receptor, es decir, un dispositivo envía datos y el resto los reciben. En un primer momento ningún extremo de la conexión tiene forma de llegar al emisor porque no se conocen entre sí. Para intercambiar entre ellos es necesario que establezcan una conexión *WebSocket* con un servidor conocido por ambas partes. Cuando el emisor se conecte al servidor se registrará con su rol concreto de transmisor y quedará a la espera. El receptor al conectarse hace una petición. El propio servidor se encarga de que esta petición llegue al emisor. Entonces el transmisor, al recibir esta solicitud, prepara un mensaje SDP (*Session Description Protocol*) con información sobre el tipo de conexión que quiere establecer y el servidor se encarga de hacérselo llegar al receptor. Cuando el emisor envía el SDP, entonces empieza un proceso llamado *ICE gathering*, en el cual busca diferentes rutas que podría utilizar el receptor para llegar hasta él. Paralelamente, el receptor, al recibir el mensaje SDP, este prepara otro mensaje SDP para el emisor, donde confirmará las propuestas de conexión del transmisor y facilitará cierta información importante. En el momento de enviar su SDP, el receptor también empezará el proceso *ICE gathering*.

En el transcurso de encontrar todas las posibles rutas se consideran tres tipos de candidatos diferentes: *Host*, *Server-Reflexive* (srflx) y *Relay*. El candidato Host es la IP privada del propio dispositivo. Esta información la ofrece directamente el sistema operativo. El candidato Server-Reflexive es la IP y puerto públicos del dispositivo. Esta información debe de ser obtenida mediante la consulta a un servidor STUN, ya que el sistema operativo no tiene acceso a las tablas NAT del *router* de su red local. Finalmente, el candidato *Relay* es un servidor *TURN*. Existe para el caso en el que no se haya podido establecer una conexión directa entre emisor y receptor. Puede suceder en caso de que en algún extremo de la comunicación haya una NAT simétrica, ya que esta cambia el puerto público por cada IP pública de destino diferente. Por lo tanto, el puerto público de la conexión con el servidor STUN no servirá para la conexión con el emisor o receptor en este caso.

A medida que van encontrando candidatos, estos son enviados directamente al servidor intermediario para que sean recibidos por el otro extremo de la conexión. De esta forma, en el momento en el que cada extremo empieza a recibir estas rutas para llegar al destino, comienza un proceso llamado *Connectivity Checks*. En este se prueban las diferentes rutas posibles hasta encontrar una que sea válida y se confirme en ambas partes. Un hecho importante es que la mayoría de NATs son *Restricted-cone*. Esto significa que solo aceptan tráfico de red de direcciones IP públicas a las que previamente algún dispositivo de la red local haya enviado algo. Por lo tanto, para que un paquete mandado por el emisor, a través de la ruta con origen en su IP pública y destino en la IP pública del receptor, sea aceptado por la NAT de este, es necesario que previamente el receptor haya enviado un paquete a la IP pública del emisor para abrir un hueco en la NAT. Como ambos lados prueban posibles rutas en bucle constantemente, llega un momento en el que tanto un extremo como el otro han enviado paquetes por una ruta probada ya en ambas partes, y, como consecuencia, las NATs quedan abiertas y permiten el paso de los paquetes por ese camino. En el momento en el que tanto emisor como receptor aceptan una ruta concreta, empieza la transmisión de datos directamente entre las dos partes sin intermediarios. Paralelamente, en caso de que surja una nueva ruta que sea mejor, se pasaría la conexión al nuevo camino.

3.7 HTTP

HTTP (*HyperText Transfer Protocol*) es un protocolo de comunicación de la capa de aplicación, basado en un modelo cliente-servidor y en un ciclo de petición-respuesta: el cliente (el navegador) envía una petición a una URL concreta, y el servidor ejecuta la acción solicitada por esa petición.

En este proyecto los dos tipos de peticiones más abundantes en las conexiones con el servidor *Web* son del tipo *GET* (pedir datos) y *POST* (enviar datos para que el servidor haga una determinada acción sobre ellos).

Este protocolo tiene ciertas limitaciones. El servidor no puede transmitir nada al usuario si no se le había solicitado información previamente.

Para dar solución a esto se creó *WebSocket*. En este protocolo la comunicación puede ser bidireccional, es decir, de cliente a servidor y de servidor a cliente, sin necesitar que la otra parte haya hecho una solicitud previamente.

3.8 MQTT

MQTT es un protocolo de mensajería ligero basado en el patrón publicación/suscripción, diseñado principalmente para dispositivos con pocos recursos. A diferencia de una comunicación directa entre dos extremos, en MQTT ningún dispositivo se conecta directamente con otro: todos interactúan a través de un intermediario común, el bróker, que se encarga de encaminar cada mensaje publicado en un *topic* concreto hacia todos los dispositivos suscritos a ese mismo *topic*, sin que emisor y receptor necesiten conocerse ni estar directamente conectados.

En este proyecto, MQTT es el mecanismo de comunicación que desacopla completamente a *Unity* de la *WebApp*, permitiendo que ambas partes lleven a cabo su actividad en redes distintas sin depender de una conexión directa entre ellas: cada una se conecta al mismo bróker público (*broker.hivemq.com*), *Unity* publicando la telemetría del dron y su estado de conexión, y la *WebApp* publicando las instrucciones de vuelo que el usuario solicita desde la interfaz de esta. Se han aprovechado además dos características propias del protocolo para dotar al sistema de mayor robustez: los mensajes retenidos, que entregan de inmediato el último valor conocido de un *topic* a cualquier nuevo suscriptor, y el mecanismo de "última voluntad" (*Last Will*), mediante el cual el propio bróker avisa automáticamente de que *Unity* se ha desconectado de forma abrupta, sin que la *WebApp* tenga que esperar un aviso explícito que, en este caso, nunca llegaría a producirse.

CAPÍTULO 4: EXPLICACIÓN FUNCIONAL DE LA APLICACIÓN

Este apartado va a ser una guía de usuario en la que se va a explicar el funcionamiento de la aplicación.

4.1 Menú Inicio

La primera pantalla que aparece al ejecutar el juego es el menú de inicio (ver figura 4.1). En este hay dos botones. Si se aprieta “Iniciar”, la aplicación redirigirá al usuario al siguiente menú. Si se selecciona salir, se cerrará el juego.

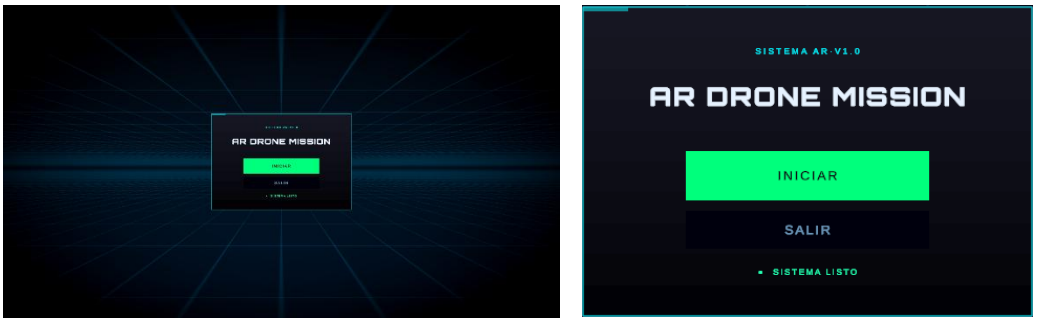


Figura 4.1: Menú de inicio.

4.2 Menú selección de misión

La siguiente escena es el menú de selección de Misión (ver figura 4.2). En esta aparecen todos los juegos existentes de la aplicación. Se observan el juego de *Disparar al blanco*, los laberintos y las pruebas. Las pruebas son un conjunto de misiones creadas desde el editor en las que hay diferentes circuitos con diferentes diseños y objetos. Con la rueda del ratón se puede bajar o subir la barra de desplazamiento vertical para poder ver todos los escenarios que hay en el menú.



Figura 4.2: Menú de selección de Misión.

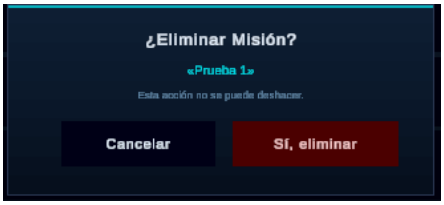
Si se selecciona una misión hecha desde el editor, aparecen dos botones en la parte derecha (ver figura 4.3). En uno está escrita la palabra editar con el emoticono de un pincel. Si se pulsa este, se abrirá el menú de edición para que se puedan hacer las modificaciones que se crean convenientes.



Figura 4.3 Botones de edición y eliminar misión.

El otro pulsador sirve para eliminar esa misión. Si es pulsado, aparecerá una ventana en la que el programa pedirá al usuario si realmente desea eliminar el escenario (ver figura 4.4a). En caso de confirmar la acción, la misión será borrada definitivamente del juego. También se puede acceder a estos botones mediante un atajo con clic derecho del ratón sobre una misión editable (ver figura 4.4b). El resto de los juegos que no están hechos desde el menú de edición no pueden ser borrados ni modificados.

a)



b)



Figura 4.4: En la imagen a) se observa la ventana de eliminar misión y en la b) se visualiza el atajo para entrar al menú de edición o para borrar el escenario.

4.3 Editor de misiones

Si se pulsa el botón “Volver” de arriba a la izquierda del menú de selección, la aplicación volverá al menú de inicio. Si se selecciona el botón “+Nuevo”, se abrirá el menú de edición para crear misiones (ver figura 4.5).

Esta pantalla es una interfaz gráfica que consiste en un mapa satelital de todo el mundo y un panel lateral en el que están todas las herramientas de edición. Para que todas las funciones operen correctamente es necesario tener conexión a internet.

Para desplazarse por el mapa hay varias opciones. La primera es mantener pulsado el botón derecho sobre el mapa y mover el ratón. La segunda es mantener pulsada la rueda del ratón y mover el *mouse*. Para aumentar o disminuir zoom hay que girar la rueda del ratón.



Figura 4.5:Menú de edición de misiones.

Se puede augmentar o disminuir el tamaño del panel de herramientas arrastrando una barra situada entre el mapa y el propio panel (ver figura 4.6).

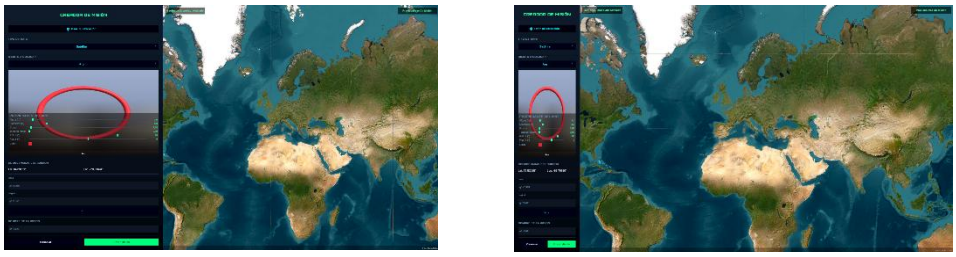


Figura 4.6 Disminución y aumento del tamaño del panel de herramientas.

El primer botón del panel tiene escrito “Usar mi ubicación” (ver figura 4.7). Al pulsarlo aparecerá un emoticono encima de la posición geográfica aproximada en la que se encuentra el ordenador del usuario.



Figura 4.7: Botón “Usar mi ubicación” para indicar posición geográfica del usuario.

Si se selecciona el pulsador “Tipo de mapa” aparecerá un desplegable con todos los diferentes tipos de filtros disponibles (ver figura 4.8).

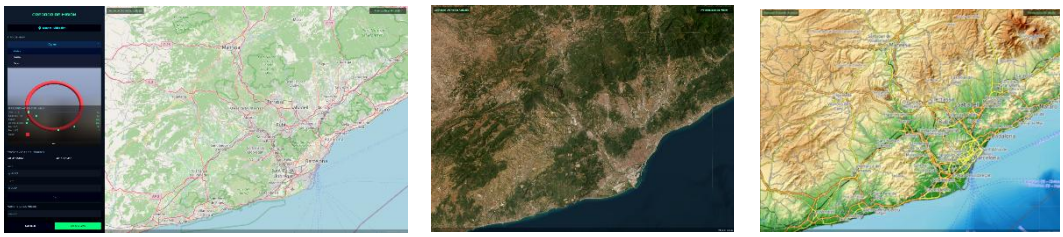


Figura 4.8: Filtros disponibles para el mapa del mundo en el menú de edición.

La siguiente sección es el previsualizado de objetos. En esta se podrá elegir el objeto que se quiera colocar en la misión y visualizarlo.

Cada objeto tiene sus propias propiedades que pueden ser modificadas (ver figura 4.9). En el caso del aro se puede cambiar la altura (en metros) a la que se quiere colocar el objeto dentro del escenario, el diámetro (en metros), el grosor, el tamaño de la abertura interior, la orientación y el color. Para cambiar estos valores se puede usar la barra deslizante o clicar encima del número directamente y escribir con el teclado el valor deseado.

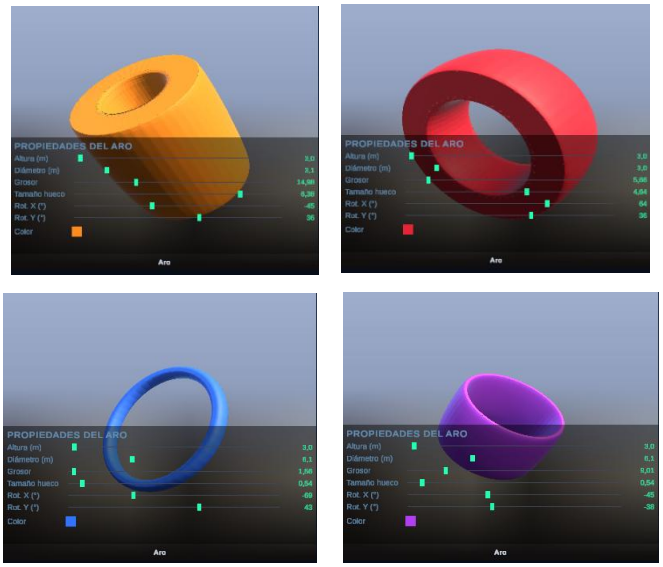


Figura 4.9: Previsualizado de objetos con los diferentes parámetros disponibles para modificar.

Además del objeto aro, el usuario puede diseñar circuitos y escenarios con banderas, obstáculos, arcos, flechas, etc. y cambiar diferentes propiedades de cada uno de estos elementos según las necesidades de este (ver figura 4.10).

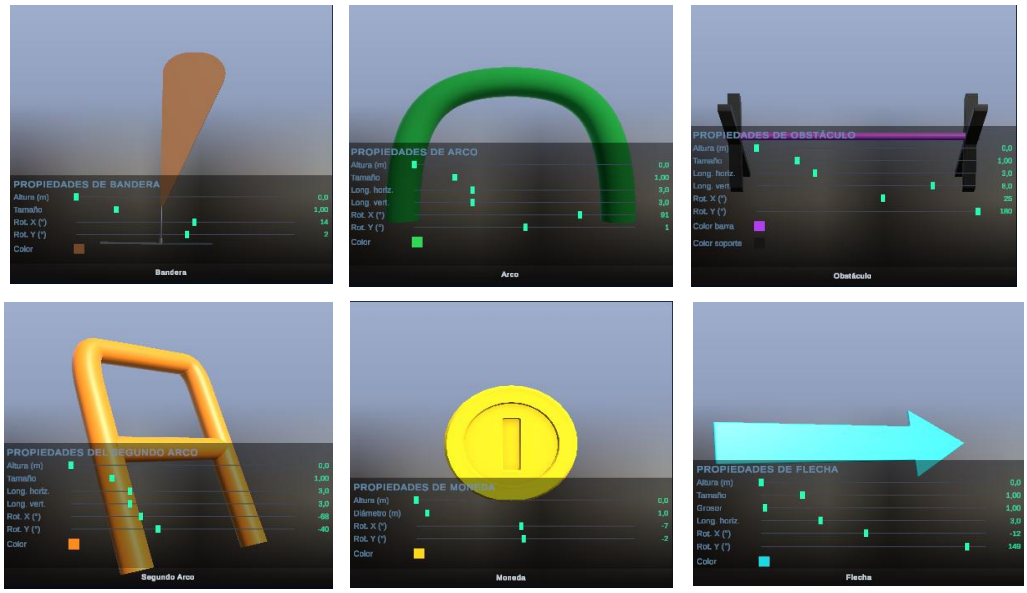


Figura 4.10: Imágenes de todos los elementos disponibles en el menú de edición.

La moneda tiene la propiedad de que si es colocada en un escenario y el dron la atraviesa, esta desaparecerá.

En caso de que se quiera colocar un objeto en alguna geolocalización del mapa, se tiene que seleccionar el elemento deseado en el previsualizado y posteriormente hacer clic izquierdo con el ratón sobre la posición del mapa en la que se quiera colocar el objeto (ver figura 4.11).



Figura 4.11 Emoticono del objeto *Aro*.

Cuando se haya ubicado el elemento, aparecerá un emoticono encima del mapa que va a tener una flecha y dos aros La flecha sirve para saber la orientación que tiene el objeto en el escenario. Los dos aros sirven para diferenciar el tipo de objeto colocado y el color de este. Concretamente el aro más externo indica el tipo de elemento que se ha posicionado. El color rojo está ligado al objeto aro, el negro a la bandera, el verde al obstáculo, el lila al arco, el azul al segundo arco y el gris a la flecha. El aro interno es del color del propio objeto (ver figura 4.12).

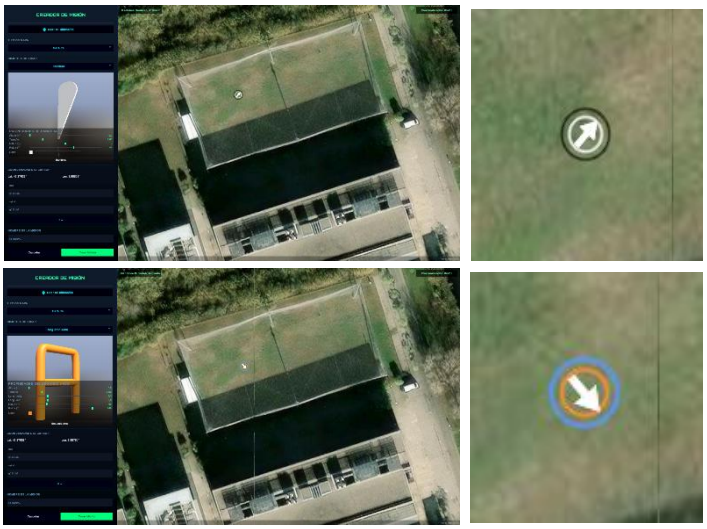


Figura 4.12:Diferentes ejemplos del color de los aros que aparece en los emoticonos.

Si se hace clic derecho con el ratón sobre un emoticono, aparecerá la opción de borrar el objeto, moverlo o copiarlo (ver figura 4.13). Si se selecciona la opción de copiar, se creará una réplica exacta con las mismas propiedades y se podrá posicionar en el sitio que desee el usuario haciendo doble clic derecho con el ratón.



Figura 4.13: Ventana de copiar, borrar o mover emoticono.

Para poner un conjunto de elementos alineados, primero hay que colocar como mínimo dos objetos y acercar uno al otro. Entonces el programa mostrará unas líneas al usuario para facilitar la alineación (ver figura 4.14). También hay un ajuste automático en el cursor del ratón cuando este está cerca de estas líneas.

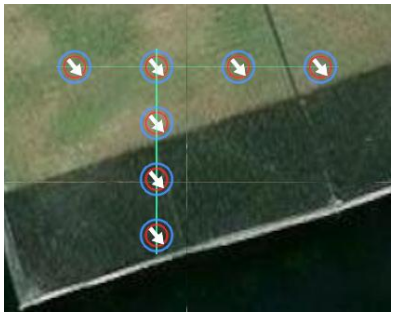


Figura 4.14: Líneas de ayuda para colocar objetos alineados.

Los elementos pueden estar encima de una línea equiespaciados fácilmente, ya que también hay un ajuste automático del cursor para esta función.

Si se pulsa clic izquierdo sobre una línea, esta se puede mover por el mapa. Con “clic izquierdo + shift” se puede rotar (ver figura 4.15).



Figura 4.15: Desplazamiento y rotación de una línea de objetos.

Si se pulsa clic derecho, aparecerán las opciones de copiar la línea entera o eliminarla. Si se elige la opción de copiar, para pegar la línea en una posición habrá que apretar dos veces seguidas clic derecho sobre la posición en la que se quiera situar la línea copiada (ver figura 4.16).



Figura 4.16: Copias de una línea de elementos.

En la esquina superior izquierda del mapa aparece un botón en el que hay escrito “Geofence Dronlab: Activado”. Este sirve para activar o desactivar las *geofences* en el recinto que ocupa el *dronlab*.

En la esquina superior derecha aparece el botón “Previsualización Misión”. Al seleccionarlo, se abrirá una nueva ventana en la que se verá un escenario que representará la zona del mapa que aparece en pantalla en ese momento (ver figura 4.17). Si se ha hecho poco *zoom* en el mapa, el área de previsualización será muy grande. Por este motivo, con tal que se vean bien los objetos hace falta hacer suficiente *zoom* para que la superficie no sea demasiado grande y no cueste observar los elementos. Para moverse por el escenario de previsualización se gira la rueda del ratón para hacer *zoom*, se pulsa la rueda del *mouse* para cambiar la orientación de la cámara y se hace clic derecho para desplazar el punto de mira lateralmente.

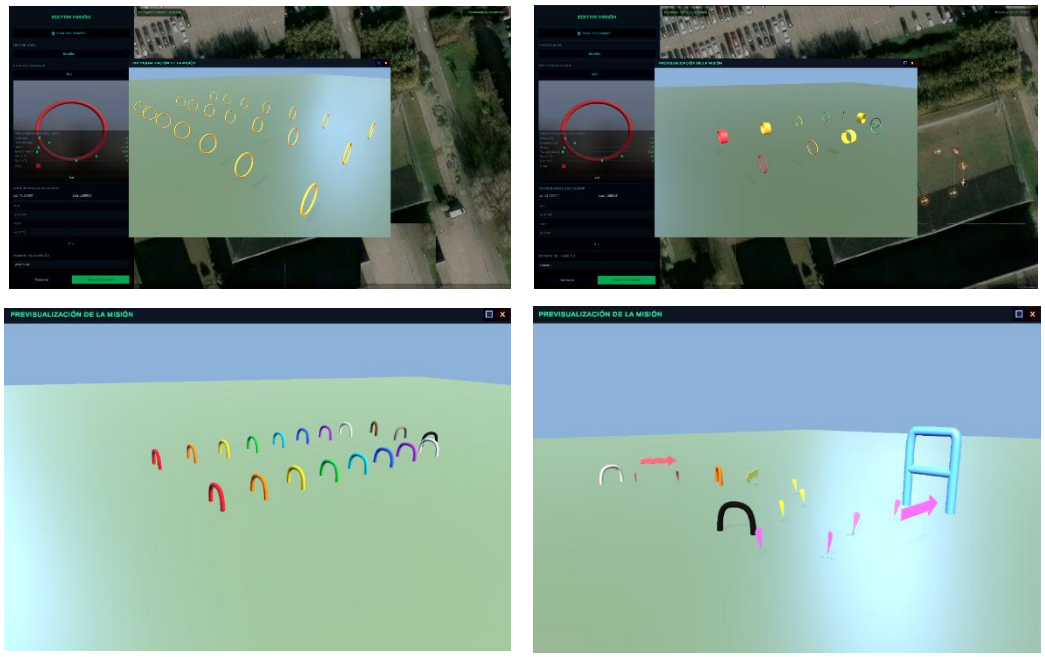


Figura 4.17: Imágenes en las que se observa el previsualizado de misiones.

La siguiente sección se llama coordenadas del cursor (ver figura 4.18). En esta aparecerán las coordenadas longitud y latitud de la posición que esté ocupando el cursor encima del mapa en ese instante. También se puede señalar un punto concreto en el mapa introduciendo las coordenadas directamente y pulsando el botón “Ir”.

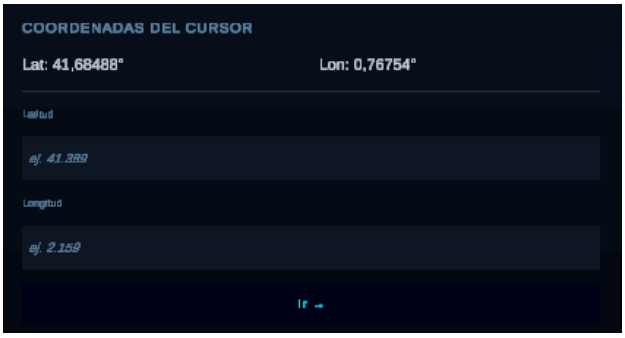


Figura 4.18: Selector de coordenadas del cursor.

Finalmente, la última sección es la llamada “NOMBRE DE LA MISIÓN” (ver figura 4.19). Esta permite al usuario elegir el título que tendrá la misión en el menú de selección de escenarios. Una vez introducido este dato, para guardar la misión habrá que pulsar el botón “Crear Misión”.



Figura 4.19: Casilla para elegir el nombre de la misión.

4.4 Menú configuración de misión

Cuando se selecciona una misión en el menú de selección y se pulsa el botón “Iniciar”, aparece el menú de configuración (ver figura 4.20). En este se pueden elegir el tipo de conexiones que se desean establecer para jugar al juego seleccionado. Si se desea jugar en modo simulación, hay que seleccionar la opción de SITL en “CONEXIÓN AL DRON” para conectarse al servidor de *Mission Planner*.

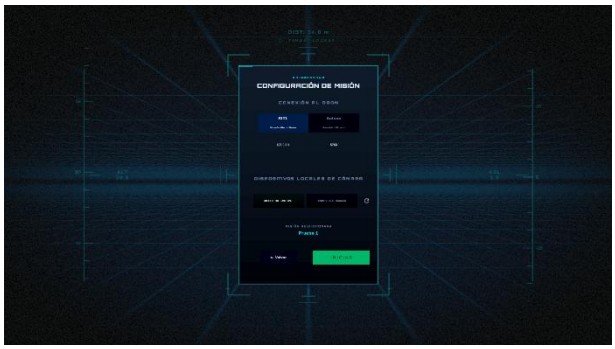


Figura 4.20: Interfaz del menú de configuración de misión.

En caso de querer conectarse al dron real mediante antena de telemetría, hay que seleccionar la opción de “Antena”. Una vez elegida esta opción, se podrá elegir entre los puertos COM en los que haya una conexión hecha. Hay que elegir el puerto en el que esté conectada la antena. En la parte derecha hay un botón de actualizar, para refrescar las conexiones en caso de que se haya conectado el puerto COM posteriormente a abrir el menú de configuración (ver figura 4.21).

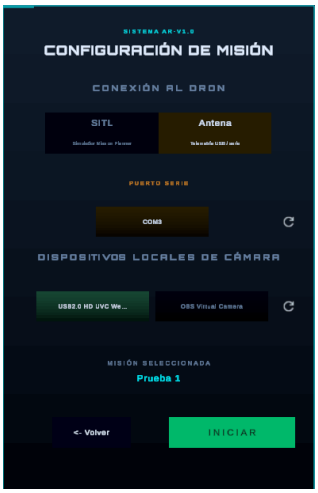


Figura 4.21: Configuración seleccionada para jugar en modo producción.

En “DISPOSITIVOS LOCALES DE CÁMARA” se puede elegir la fuente de vídeo que se desea tener para obtener el vídeo real que se observará en el juego. En “MISIÓN SELECCIONADA” se observa el escenario que se ha elegido en el menú de selección. Finalmente, si el usuario ya ha realizado la configuración, puede seleccionar el botón “INICIAR”. En caso de querer cambiar de misión, se puede seleccionar el botón “Volver” para retroceder al menú de selección (ver figura 4.21).

4.5 Juegos

Posteriormente a este menú, la estructura de las siguientes pantallas cambia según el juego elegido.

4.5.1 Juego creado en el editor

En caso de elegir un escenario creado desde el editor, cuando se seleccione el botón “INICIAR” en el menú de configuración se ejecutará el juego directamente. Los elementos en realidad aumentada se observarán solo si el dron real o el simulado se encuentra en la zona geográfica en la que se habían posicionado esos objetos en el editor de misiones. En el caso de todos los escenarios “prueba” que vienen con el ejecutable de la aplicación, los objetos están situados dentro del *dronlab con* las *geofences* activadas, aunque se pueden desactivar desde el menú de edición. Cuando se haya cargado la pantalla, ya se podrá ver el juego y el HUD de vuelo (ver figura 4.22). El *Head-Up Display* es una interfaz de vuelo situada en la pantalla del jugador que permite al piloto visualizar datos importantes en tiempo real. En la barra izquierda se muestra la altitud a la que vuela el dron en metros. La banda de la derecha indica la velocidad sobre el suelo (*Ground Speed*) a la que está volando el dron en metros por segundo. La franja superior es una brújula que señala el rumbo de vuelo en ese momento en grados. La barra central es una mira que no tiene uso en este juego. Las líneas que rodean a esta son indicadores del ángulo de cabeceo del dron (*pitch angle*) en grados.

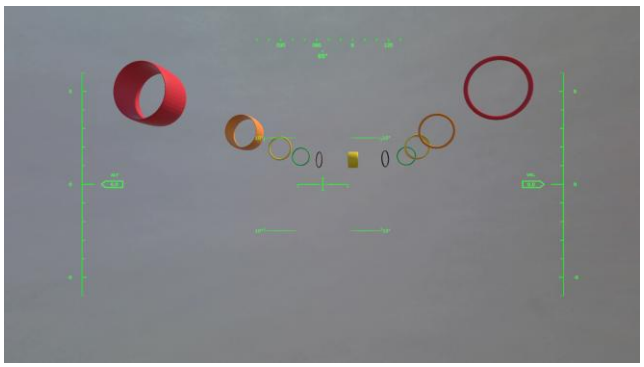


Figura 4.22: Imagen de la escena del juego “Prueba 3” con el HUD de vuelo activado.

En este juego y en todos, pulsando la tecla escape del teclado del ordenador se pausa la aplicación (ver figura 4.23). Desde esta pantalla se puede volver al menú de inicio, reanudar el juego, cambiar el color del HUD a blanco o negro (cambiar de color evita que si el fondo es muy claro u oscuro se pierda visibilidad de la interfaz), o desactivarlo directamente.

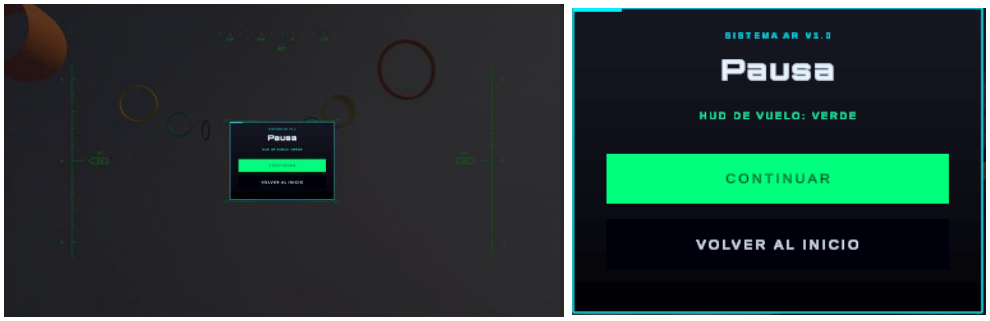


Figura 4.23: Interfaz del menú de pausa.

4.5.2 Laberintos

Los siguientes juegos son los laberintos. Estos también se ejecutan directamente al pulsar el botón “INICIAR” desde el menú de configuración. Tienen el mismo HUD que los juegos hechos desde el editor. Los tres laberintos que vienen con la aplicación están situados dentro del recinto del *dronlab*. Estos juegos tienen la particularidad de que no solamente hay *geofences* en los muros del recinto, sino que también hay en las paredes de los laberintos. Por lo tanto, cuando el jugador intente atravesarlas, el dron frenará automáticamente (ver figura 4.24).

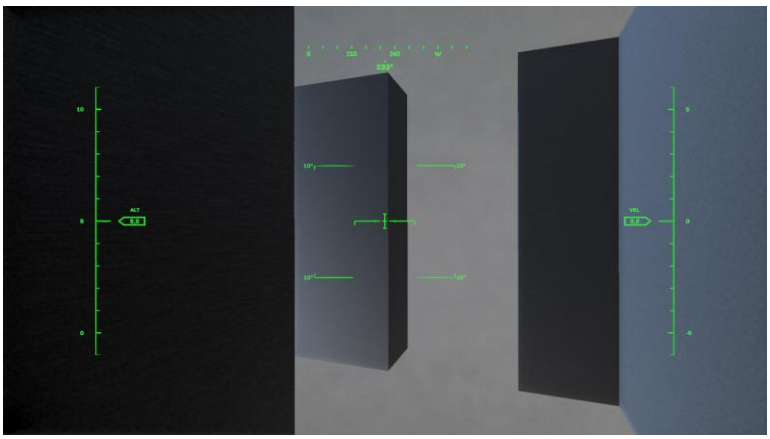


Figura 4.24: Imagen de la escena “Laberinto 2”.

La finalidad del juego es encontrar la salida. En el suelo hay flechas que indican el camino para escapar del laberinto (ver figura 4.25).

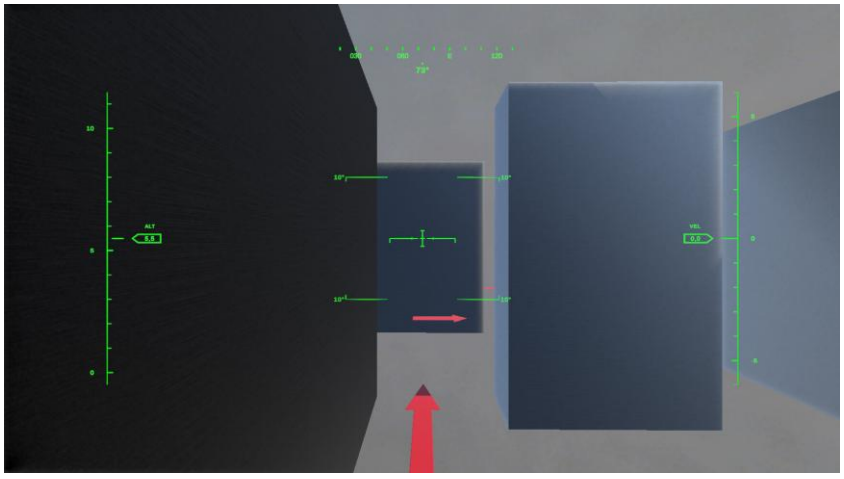


Figura 4.25: Flechas orientativas del juego “Laberinto 2”.

4.5.3 Disparar al blanco

El último juego es “Disparar al blanco”. En este al seleccionar el botón “INICIAR” en el menú de configuración, se abre un nuevo menú de ajustes (ver figura 4.26). El juego se ubica en el *dronlab* y consiste en eliminar enemigos que disparan y se mueven de forma aleatoria dentro del recinto del videojuego. En esta pantalla de configuración se puede elegir primeramente el tipo de enemigo contra el que se va a enfrentar el usuario.

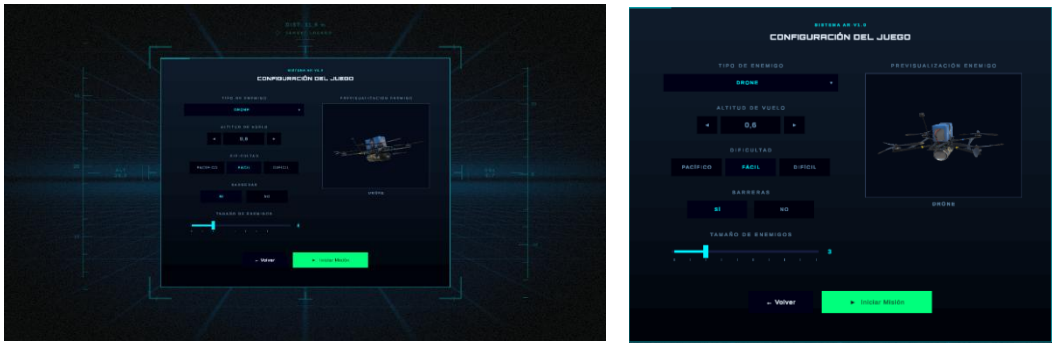


Figura 4.26: Interfaz del menú de configuración del juego “Disparar al blanco”.

El siguiente parámetro modificable es la altura en la que se generarán los enemigos. Durante el transcurso del juego, estos tienen una altura fija. El usuario también puede elegir el nivel de dificultad. Existe el modo pacífico, fácil y difícil. En el modo pacífico no te disparan. En modo fácil, el comportamiento de los enemigos al inicio es menos agresivo que en modo difícil. Al principio hay en total 3 adversarios. Cada vez que se elimina uno, se genera otro. Cada 5 derribos se genera un enemigo adicional, y el nivel de agresividad de estos incrementa con el aumento de algunos parámetros, como pueden ser la cadencia de disparo, la precisión, la velocidad de movimiento y la probabilidad de esquive. El máximo número de enemigos que puede haber en el escenario es 30, y estos no pueden disparar al usuario si están a más de 15 metros de distancia del jugador. La vida inicial que tiene el piloto es de 100 puntos. En modo fácil los disparos quitan 5 puntos de vida al jugador, mientras que en modo difícil 10. Los enemigos son eliminados de un solo disparo.

Este menú de configuración también permite elegir al usuario si desea *geofences* en el *dronlab* o no. Finalmente, el último parámetro modificable es el tamaño de los enemigos. Cuanto más grandes sean estos más fácil será dispararlos.

El HUD de vuelo tiene ciertas diferencias con respecto al del resto de juegos de la aplicación (ver figura 4.27). Arriba a la izquierda aparece una barra que indica la vida que tiene el jugador. En la parte superior derecha aparece un indicador con el número de derribos que ha realizado el usuario. A la derecha de este aparece el arma que tiene seleccionada el jugador en ese momento. Hay dos tipos diferentes, el *LASER* y el *MISSILE*. Para cambiar entre ellas hay que hacer clic derecho con el ratón.



Figura 4.27: HUD de vuelo del juego “Disparar al blanco”.

Los enemigos salen designados con un rombo (ver figura 4.28). Cuando aparecen en pantalla, primero aparece el indicador en color amarillo con el texto “ACQUIRING TARGET”.

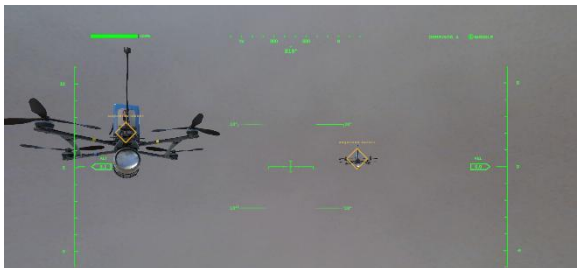


Figura 4.28: Imagen del juego “Disparar al blanco” con dos enemigos designados.

Unos instantes después el color del designador se vuelve rojo. Cuando se da esta transición, aparece el texto “TARGET LOCKED” y debajo del rombo aparece indicada la distancia a la que está el enemigo en metros (ver figura 4.29).

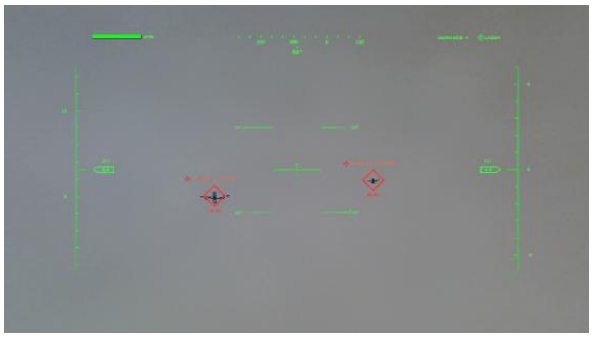


Figura 4.29: Designador de los enemigos en rojo.

Cuando el jugador dispara el láser haciendo clic izquierdo con el ratón, el dron lanza un proyectil dirigido hacia el centro de la mira (ver figura 4.30).



Figura 4.30: Proyectil *LASER* disparado por el jugador.

Para usar el arma *MISSILE*, primero se tiene que esperar a que el designador del enemigo se ponga de color rojo. El disparo de esta arma a diferencia del láser es autoguiado, y cuando se dispara, el dron lanza un proyectil que sigue al enemigo que haya seleccionado el jugador. El usuario para cambiar de objetivo debe seleccionar las flechas laterales del teclado. El enemigo seleccionado aparecerá con un recuadro alrededor del designador (ver figura 4.31).

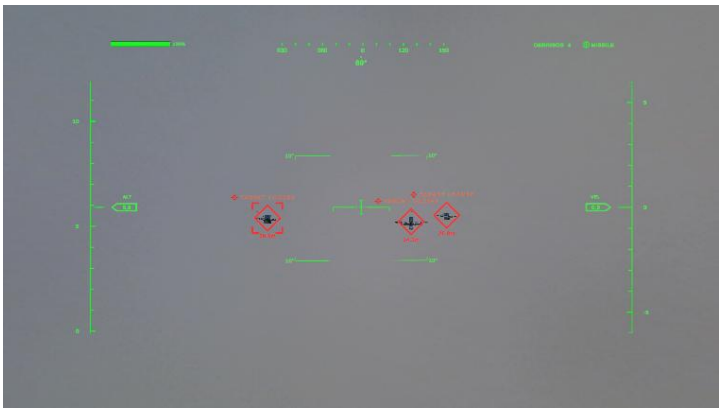


Figura 4.31: Designador del *MISSILE*.

El *MISSILE* cada vez que es disparado tiene una recarga de 30 segundos (ver figura 4.32). El tiempo restante para que vuelva a estar disponible es mostrado al lado del indicador de arma seleccionada.

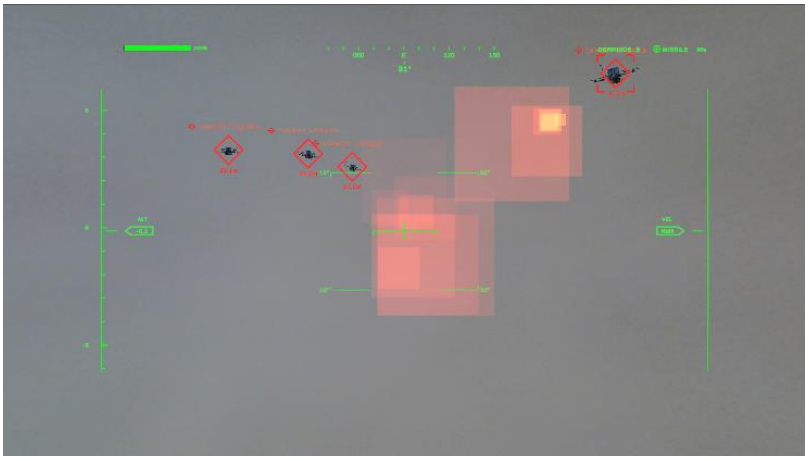


Figura 4.32: Proyectil *MISSILE* disparado al objetivo.

4.6 WebApp

Para conectarse a la *WebApp* a través de un navegador se debe escribir en el buscador la dirección IP y puerto en el que se esté ejecutando el servidor web. La primera pantalla que aparecerá al conectarse le pedirá al usuario que seleccione el método de conexión al dron (ver figura 4.33).

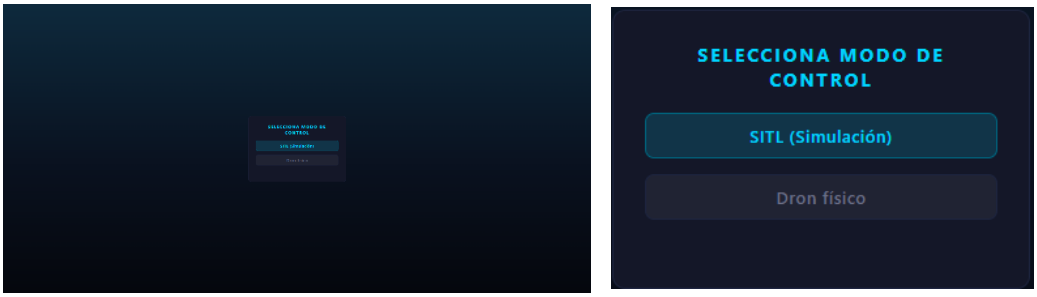


Figura 4.33: Pantalla de inicio de la *WebApp.*

Si se elige la opción “Dron físico” aparecerá una lista con todos los puertos COM conectados que haya en el ordenador en el que se esté ejecutando la aplicación de Unity (ver figura 4.34). Hay que elegir el puerto COM al que esté conectada la antena de telemetría.

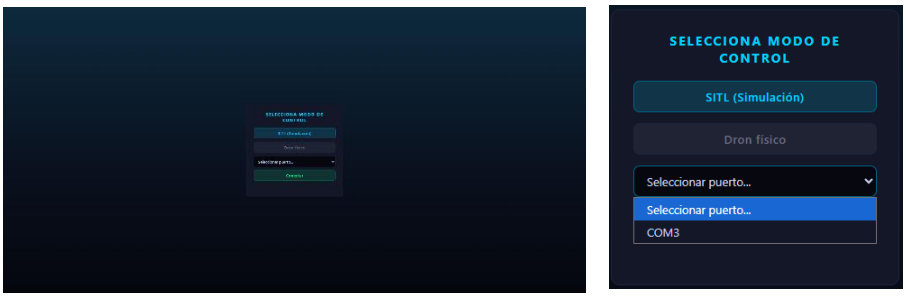


Figura 4.34: Selección del puerto COM al que esté conectada la antena de telemetría.

Una vez establecida la conexión, aparecerá una pantalla con el vídeo del juego y dos *joysticks (*ver figura 4.35). Estos últimos permiten al usuario controlar el dron desde el dispositivo con el que se haya conectado a la *WebApp*.

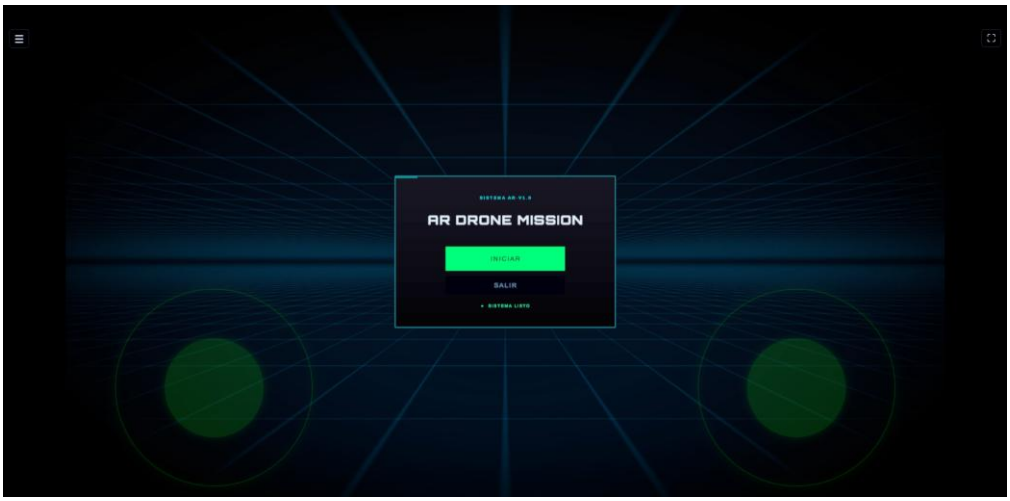


Figura 4.35: Interfaz de la *WebApp* con el juego ejecutándose desde *Unity*.

En la parte de arriba a la derecha hay un pulsador que sirve para maximizar o minimizar la pantalla. El botón de la zona superior izquierda despliega un panel lateral con distintas herramientas (ver figura 4.36). Desde esta ventana se pueden activar o desactivar los *joysticks*, cambiar el color de estos, elegir entre diestro o zurdo para cambiar las funciones de cada *joystick*, modificar la sensibilidad de estos y bloquear el empuje para evitar que se pueda cambiar de altura con los controles. Con respecto al vuelo, se pueden enviar las siguientes instrucciones: armar el dron, despegar, aterrizar, cambiar el modo de vuelo y volver al punto inicial.

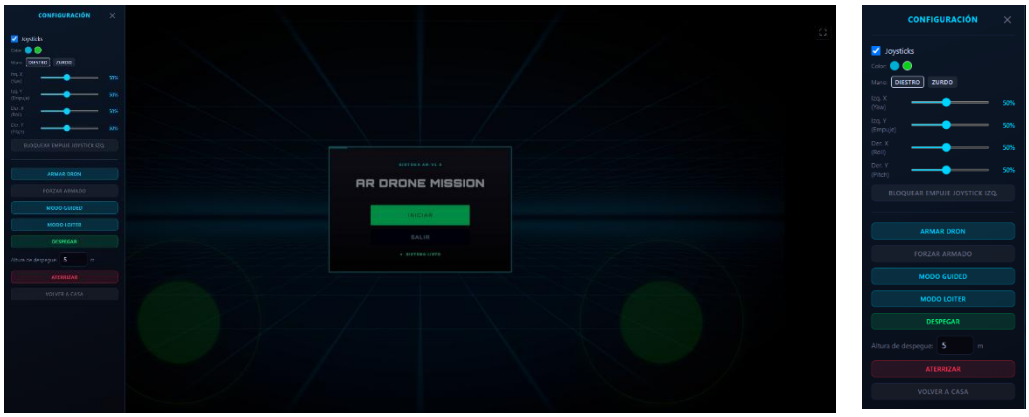


Figura 4.36: Menú desplegable de la *WebApp*.

En el juego de “Disparar al blanco”, aparecen tres botones más en la interfaz. A la derecha del *joystick* derecho se sitúa el botón de disparar de color rojo. En la parte superior derecha aparece el botón de cambiar de arma. Debajo de este se observa el pulsador para cambiar de objetivo (ver figura 4.37).

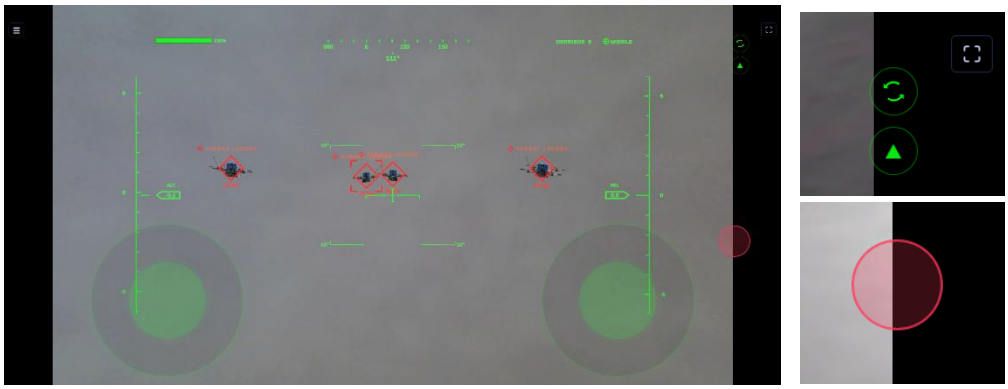


Figura 4.37: Interfaz de la *WebApp* en el juego “Disparar al blanco”.

CAPÍTULO 5: ARQUITECTURA SOFTWARE

En este apartado se va a realizar un análisis de la arquitectura *software* que tiene tanto la aplicación AR de *Unity* como la *WebApp*. Se van a exponer diferentes extractos del código fuente con el fin de mostrar de una forma más profunda el funcionamiento de las diferentes aplicaciones.

5.1 Menú inicio

En el escenario *inicio* aparecen los siguientes elementos en el *Hierarchy* (ver figura 5.1).

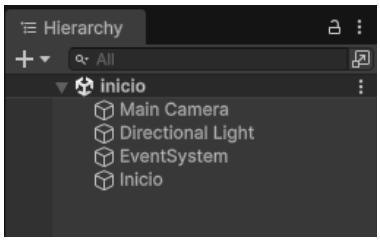


Figura 5.1: *Hierarchy* del escenario *Inicio*.

El primero es la *Main Camera*. Esta no se usa como tal en esta escena, ya que la pantalla de inicio es un *Canvas* en modo *ScreenSpaceOverlay*. Esto implica que este se dibujará sobre la pantalla de la aplicación cuando se ejecute. Por lo tanto, la imagen de la escena captada por la *Main Camera* no será visible para el usuario final. La *Directional Light* es una fuente de luz proveniente de un punto específico del escenario. Tampoco es un objeto que afecte a la visualización de la pantalla de inicio, pero siempre viene predeterminado en cualquier escena que se cree. Se podría eliminar sin ninguna consecuencia, igual que la *Main Camera*. El *Eventsystem* es el objeto encargado de detectar la posición de la pantalla del juego en la que está el cursor del ratón y detectar si ha sido pulsado o no. Por ejemplo, si se ha pulsado encima de un botón que hay en el *Canvas*, el *EventSystem* escaneará todos los elementos de la pantalla utilizando los métodos de la clase pública *GraphicRaycaster* y detectará sobre qué zona del *Canvas* se ha pulsado. En caso de que haya un botón en esa localización, llamará a la función *OnPointerClick()* del propio botón en cuestión para indicarle que ha sido pulsado.

Finalmente está el objeto *Inicio*. Este tiene adjunto el script *Inicio.cs*. Esto significa que se ha arrastrado el script desde la pestaña *Project* hasta el objeto *Inicio* del *Hierarchy*. Cuando se hace esto, automáticamente el script aparece en el inspector del objeto al que ha sido adjuntado (ver figura 5.2).

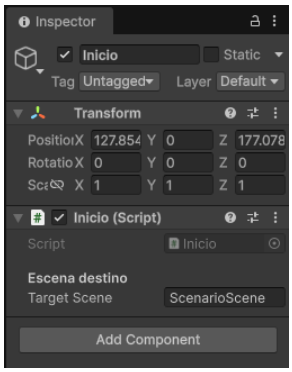


Figura 5.2 *Inspector* del objeto *Inicio*.

Este script contiene el código que generará el *Canvas* con sus diferentes elementos. Se ejecuta en el momento en el que se crea la escena. Esto se debe a que el motor de *Unity* llama automáticamente a diferentes funciones de los scripts que están adjuntos a algún *GameObject* del *Hierarchy*. Este es el caso del método *Start()* de *Inicio.cs* (ver tabla 5.1).

Función Start() de Inicio.cs

```
Void Start()
{
        RemoveExistingCanvas();
        BuildUI();
}
```

Tabla 5.1 Función *Start()* del script *Inicio.cs*.

Dentro del script se crea también el fondo animado del *Canvas*. El *shader* que procesa los píxeles de este fondo es el llamado *ARDashboard_Background_39*. Los *shaders* necesitan asociarse a un objeto propio de *Unity* llamado *Material*. Este sirve para aplicar un aspecto visual concreto a un elemento de la escena, como puede ser un aro, una flecha, etc. El *Material* que se va a asociar a este *shader* se crea en el script *inicio.cs* mediante la función *GetMaterial()* del código *ShaderTimeKeeper.cs* (ver tabla 5.2). Esta función crea el *Material* que va a crear el fondo del *Canvas*.

Función GetMaterial() de Inicio.cs

```
public static Material GetMaterial()
{
        CreateRunner();
        if (_material != null) return _material;
        var source = Resources.Load<Material>("Materials/Background");
        if (source != null)
        {
                _material = new Material(source);
                TickOnceIfNotPlaying();
                return _material;
        }
        var shader = Shader.Find("ARDashboard/Background");
        if (shader != null)
        {
                _material = new Material(shader);
                Debug.LogWarning("[ShaderTimekeeper] Material creado desde shader (fallback).");
                TickOnceIfNotPlaying();
                return _material;
        }
        Debug.LogError("[ShaderTimekeeper] No se encontró Background en Resources ni el shader.");
        return null;
}
```

Tabla 5.2: Función *GetMaterial()* del script *Inicio.cs.*

Dentro de este mismo script está también la función *Tick()* (ver tabla 5.3). Este método se encarga de acceder a ese mismo *Material* y actualizar varios de los parámetros que tiene este. Entre ellos está el *ID_Timeoff*, el cual cada vez que se llama a la función *Tick()* se actualiza. Esta variable representa el tiempo transcurrido desde que se empezó a ejecutar el juego. El *shader* utiliza este valor para saber qué imagen debe construir exactamente, porque al ser un fondo animado, en cada *frame* cambia la imagen ligeramente. Esta función Tick() es llamada por la corutina *TickLoop()* del propio script *ShaderTimeKeeper.cs*. Es un tipo de función que se pausa su ejecución cada vez que se alcanza un *yield*. El motor de *Unity* llama a este tipo de funciones una vez por *frame* si se ha programado de esta forma, lo cual resulta útil para este método en concreto, ya que en cada fotograma se reanuda la ejecución de la función TickLoop(), que a la vez provoca que se actualice el valor *ID_Timeoff* para que el *shader* cree la imagen del fondo con el nuevo valor de esta variable. Esta actualización es la que produce la animación. *TickLoop()* al ser un bucle infinito, se reanuda y pausa en cada iteración de forma indefinida y no finaliza nunca.

Función Tick() de Inicio.cs

```
public static void Tick()
{
        var mat = GetMaterial();
        if (mat != null)
        {
                mat.SetColor(ID_Deep, ColorDeep);
                mat.SetColor(ID_Mid, ColorMid);
                mat.SetColor(ID_Accent, ColorAccent);
                mat.SetFloat(ID_GridSize, GridSize);
                mat.SetFloat(ID_GridAlpha, GridAlpha);
                mat.SetFloat(ID_GridSpeed, GridSpeed);
                mat.SetFloat(ID_ScanAlpha, ScannerEnabled ? ScanAlpha : 0f);
                mat.SetFloat(ID_BarAlpha, ScannerEnabled ? 0.12f : 0f);
                mat.SetFloat(ID_ScanTimeOff, ScannerEnabled ? Time.unscaledTime - _scanStartTime :
                0f);
                mat.SetColor(ID_ScanColor, ColorScan);
                mat.SetFloat(ID_PulseSpd, PulseSpeed);
                mat.SetFloat(ID_PulseCnt, PulseCount);
                mat.SetFloat(ID_Vig, VigStr);
                mat.SetFloat(ID_TimeOff, Time.unscaledTime);
        }

        var fpv = GetFPVMaterial();

        if (fpv != null)

        fpv.SetFloat(ID_TimeOff, Time.unscaledTime);

}
```

Tabla 5.3: Función *Tick()* del script *Inicio.cs*.

Para iniciar *TickLoop(),* se ejecuta la función *StartCoroutine()* dentro de la función *Start()* de *ShadertimeKeeper.cs* (ver tabla 5.4). Esta última función, como ya se ha explicado, es llamada por el motor de *Unity* en caso de que esté en un script asociado a un *GameObject* presente en la escena. En la figura 5.1 aún no se observa un objeto llamado *ShaderTimeKeeper* debido a que es una captura del juego sin arrancar, pero al ejecutar el juego, aparecerá de forma automática en el *Hierarchy* un objeto con estas características. La función *Start()* es llamada solo una vez al iniciarse la escena. De esta forma se consigue arrancar el bucle y que se actualice el valor de *ID_Timeoff* en cada *frame*.

Función Start() de Shadertimekeeper.cs

```
void Start()
{
        StartCoroutine(TickLoop());
}

IEnumerator TickLoop()
{
        while (true)
        {
                ShaderTimekeeper.Tick();
                yield return null; // avanza frame a frame, inmune a timeScale=0
        }
}
```

Tabla 5.4: Función *Start()* del script *Shadertimekeeper.cs*.

Como se observa en la figura 5.3, en el momento de ejecutar la aplicación y crearse la escena *Inicio*, aparecen objetos nuevos que han sido creados precisamente por el script *inicio.cs*. Todos los objetos que están dentro del objeto *Inicio* son los diferentes elementos del *Canvas*: los botones, la decoración, etc.

También se observa que ha aparecido la escena *DontDestroyOnLoad*. Dentro de esta aparecen los objetos que se crean en el momento de arrancar el juego y permanecen en todas las escenas sin destruirse. Se observa el *ShaderTimeKeeper*, que ya se ha comentado anteriormente, el *DronMqttBridge*, el cual tiene adjunto el script *DroneMqttBridge.cs* y es el encargado de gestionar las conexiones con otros programas fuera de *Unity*, y el *UnityVideoStreamer*, que tiene adjunto el script *UnityVideoStreamer.cs*. Este es el encargado de capturar la imagen del juego y enviarla usando la tecnología *WebRTC* a los diferentes usuarios que se hayan conectado a la *WebApp*.



Figura 5.3: *Hierarchy* del escenario *Inicio* en ejecución.

5.2 Menú selección de misión

Cuando se selecciona el botón “Iniciar” desde el menú de inicio, el juego abre automáticamente la escena *Menú*.

En el *Hierarchy* (ver figura 5.4) aparecen los mismos elementos que en *Inicio*. Todos tienen la misma función. La diferencia es que ahora en vez del objeto *Inicio*, aparece *Menú*. Este tiene adjunto el script *Menú.cs*. Tiene la misma función que *inicio.cs*, es decir, construir el *Canvas* donde van a estar todos los elementos del menú.

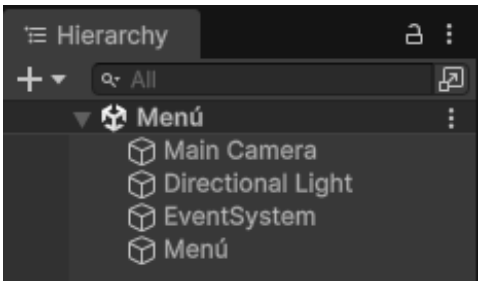


Figura 5.4: *Hierarchy* del escenario *Menú.*

5.3 Editor de misiones

Si se selecciona el botón de “+Nuevo” desde el menú de selección se abre el editor de misiones. La nueva interfaz no es creada en una escena diferente, sigue siendo la escena *Menú*. *Menu.cs* tiene un método asociado al botón “+Nuevo”, que se encarga de crear un nuevo *Canvas* con todos los elementos de la nueva interfaz. Este método es el llamado *OpenMapCreator(),* el cual a su vez invoca la función *show()* del script *MapCreatorUI.cs*, la cual contiene el código que construye todos los elementos del nuevo *Canvas*.

El script *MapCreatorUI.cs* se encarga también de invocar funciones de otro script llamado *MapSceneController.cs*. Concretamente inicia la función *GPSToWorld(),* que analiza el *zoom* y la zona en la que se ubica el mapa en ese momento. Esta información es importante de cara a crear el previsualizado de misiones. También llama a la función *PlaceAll()*, que se encarga de posicionar los objetos colocados en el mapa en el previsualizado. El script *MapSceneController.cs* tiene la función principalmente de obtener los datos de las misiones que se habían guardado desde el editor y reconstruirlas cuando se inicien para jugarlas, situando las *geofence* y colocando todos los elementos correspondientes en la escena en la ubicación previamente seleccionada en el editor. Hay otro script que participa en el menú de edición de misiones, y es el llamado *AroHuecoUtil.cs*. Este es necesario porque no hay una herramienta que tenga *Unity* para cambiar el tamaño del hueco del aro. Por lo tanto, cuando se cambia este parámetro en el previsualizado de objetos, *MapControllerUI.cs* invoca a la función *ApplyHueco()* del script *AroHuecoUtil.cs*. Esta manipula el objeto aro para conseguir que tenga el hueco deseado. Este método también es llamado en el momento de activar el previsualizado de misiones, para colocar los aros con los huecos modificados, y por el MapSceneController.cs para crear los aros con hueco que va a colocar en la escena cuando se inicie la misión para jugar.

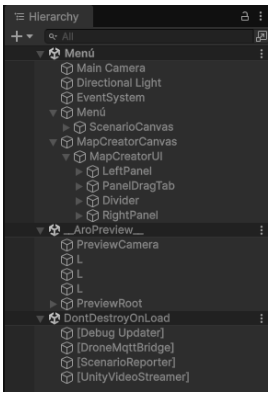


Figura 5.5: *Hierarchy* del escenario *Menú* en el editor de misiones en ejecución.

Se puede observar en el nuevo *Hierarchy* (ver figura 5.5) como al pulsar el botón “+Nuevo” aparecen estos nuevos elementos de los que se ha estado tratando. El primero observable es el objeto *MapCreatorCanvas*, en el interior del cual están todos los elementos del nuevo *Canvas*. Se observa también que ha aparecido *AroPreview*. Esta es una nueva escena que se crea en tiempo de ejecución y sirve para renderizar el objeto 3D que se muestra en el previsualizado de un objeto del menú de edición.

Si se clica el botón de “Previsualización Misión”, entonces en el *Hierarchy* aparecen nuevos elementos (ver figura 5.6). Aparece *ScenarioPreview* que hace lo mismo que *AroPreview* pero para el previsualizado de toda la misión.

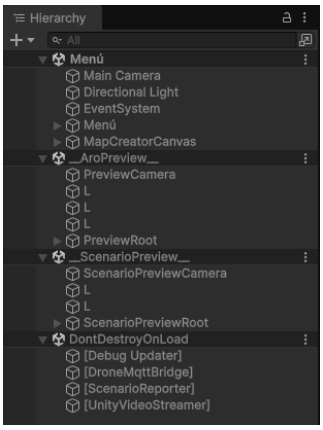


Figura 5.6: *Hierarchy* del escenario *Menú* en el editor de misiones y con el previsualizado de misión abierto.

5.4 Menú configuración

Cuando se selecciona un juego y se clica en el botón “Iniciar” en el menú de selección, aparece el menú de configuración. Esta escena tiene el siguiente *Hierarchy* (ver figura 5.7).

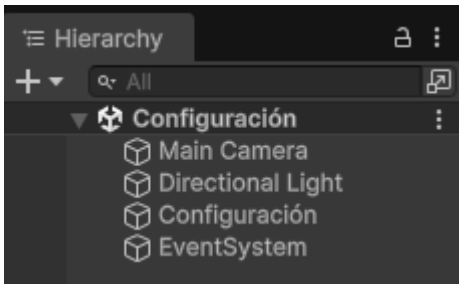


Figura 5.7: *Hierarchy* del escenario *Configuración*.

Tiene la misma estructura de objetos que las dos anteriores, con el script *Configuración.cs* adjuntado al objeto *Configuración*. Este script, aparte de tener el código que construye *el Canvas* de este menú en el momento de ejecutarse, tiene otras funciones. Concretamente guarda la configuración que se ha elegido para el juego almacenando la información en diferentes variables de la clase estática *GameConfig*. Esta clase vive en el script *GameConfig.cs*, y permite a cualquier otro script acceder a los datos de esta configuración consultando los valores de la instancia correspondiente. Una vez elegida la configuración, el juego arrancará al pulsar el botón “Iniciar”.

5.5 Juegos

Cada juego de la aplicación tiene una estructura diferente y se configuran y ejecutan de diferentes formas.

5.5.1 Juego creado en el editor

Los juegos creados en el editor no tienen escena propia, ya que se crean desde la propia escena de configuración. En el *Hierarchy* (ver figura 5.8) se observa que han aparecido objetos nuevos y se ha borrado el *Canvas* del menú. El motivo por el cual no tienen escena propia los juegos creados desde el editor se debe a que después de hacer la *build* final de la aplicación, no se pueden añadir nuevas escenas en el *archivo.exe*. Por lo tanto, la única forma de crear nuevas misiones desde el editor una vez hecha la *build* final es reaprovechando la propia escena de configuración.

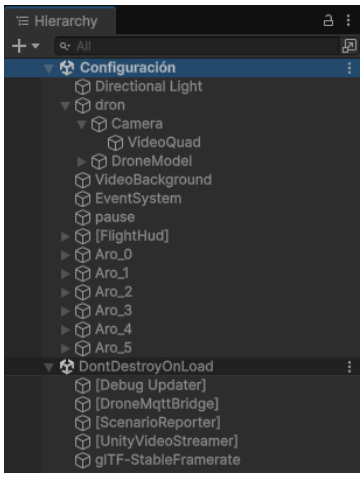


Figura 5.8: *Hierarchy* de los juegos creados desde el editor en ejecución.

El primer script importante es *DronMqttBridge.cs*, que ya se había nombrado anteriormente y que está adjunto a un objeto dentro de la escena permanente *DontDestroyOnLoad*, la cual siempre está activa mientras la aplicación esté en ejecución, independientemente de los cambios de escena. Este script es el encargado de conectarse al SITL de *Mission Planner* en caso de simulación o al puerto COM de telemetría en modo producción. También se encarga de recibir las diferentes instrucciones enviadas desde la *WebApp* para posteriormente enviarlas al dron físico o simulado a través de sus conexiones. Las peticiones las recibe desde el servidor *HiveMQ* a través del protocolo MQTT, es decir, el script se conecta al servidor público *HiveMQ* y se suscribe a ciertos *topics* para que cuando la *WebApp* envíe alguna instrucción a través de alguno de estos temas, el script *DronMQttBridge.c*s la pueda recibir. Cuando se inicia la aplicación de *Unity*, se ejecutan las siguientes funciones en este script.

Función Bootstrap() de DronMqttBridge.cs

```
 [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)] //1

 static void Bootstrap() //2
{
        var go = new GameObject("[DroneMqttBridge]"); //4
        go.AddComponent<DroneMqttBridge>(); //8
}
```

Tabla 5.5: Función *Bootstrap()* del script *DronMqttBridge.cs.*

En un primer momento *Unity* compila todo el código de C# que hay en el proyecto. Al ejecutar el juego, el motor de *Unity* escanea el archivo compilado, y en caso de leer una línea como la 1 de la tabla 5.5, lo que hace es guardar una referencia a la función *Bootstrap()* de la línea 2 para ejecutarla cuando se inicia el programa por primera vez. En este caso, se observa en la línea 4 que en esta función se crea el objeto *DroneMqttBridge*, al cual se le asocia el componente *DronMqttBridge.cs*, que es el script que se había mencionado anteriormente.

Una vez aparece este *GameObject* en escena con un script adjunto, el motor de *Unity* llama a varias funciones de este automáticamente de forma secuencial. La primera a la que llama es a la función *Awake() (*ver tabla 5.6). Esta sirve como mecanismo de seguridad para evitar crear otra instancia de la clase *DronMqttBridge* en caso de que ya exista una.

Esto se consigue declarando la variable estática *_instance*, la cual tiene el mismo valor para todas las instancias de la misma clase al ser *static*. En caso de que ya se hubiese creado una variable de este tipo anteriormente, se cumpliría la condición del *if* de la línea 3 de la tabla 5.6 al ser *_instance* una instancia diferente a la que se está ejecutando en ese momento, y se destruiría el *GameObject.*

Función Awake() de DronMqttBridge.cs

```
void Awake()
{
if (_instance != null && _instance != this) // 3
{
        Destroy(gameObject); //5
        return; //6
}
        _instance = this;
        DontDestroyOnLoad(gameObject);
}
```

Tabla 5.6: Función *Awake()* del script *DronMqttBridge.cs.*

La siguiente función que el motor de *Unity* llama de forma automática es la función *Start() (*ver tabla 5.7).

Función Start() de DronMqttBridge.cs

```
void Start()
{
        if (_instance != this) return;      //3
        _topicTelemetry     = $"dron/{droneId}/telemetry";
        _topicStatus         = $"dron/{droneId}/status";
        _topicPorts          = $"dron/{droneId}/ports";
        _topicError          = $"dron/{droneId}/error";
        _topicAvailable      = $"dron/{droneId}/available";
        _topicWeaponMode     = $"dron/{droneId}/weapon_mode";
        _topicScenario       = $"dron/{droneId}/scenario";
        _topicWebrtcRequest  = $"dron/{droneId}/webrtc_request";
        _cmdPrefix           = $"dron/{droneId}/cmd/";
        _cmdFilter           = _cmdPrefix + "+"; //13

        _messageHandlerField = typeof(csDronLink.Dron).GetField() //15

        "messageHandler", BindingFlags.NonPublic | BindingFlags.Instance); //17



        Task.Run(ConnectToBrokerAsync); //20
        StartCoroutine(TelemetryPublishLoop()); //21
        StartCoroutine(RcOverrideLoop());  //22
        StartCoroutine(StatusAndPortsLoop()); //23
        StartCoroutine(ConfigWatchLoop()); //24

}
```

Tabla 5.7 Función *Start()* del script *DronMqttridge.cs*.

La línea 3 de la tabla 5.7 es otra capa de seguridad para evitar que, si ya hay una instancia creada de la clase *DronMqttBridge*, no se ejecute la función. Podría parecer que ya es suficiente el hecho de que si en la función *Awake()* el objeto ya se destruyó no sería necesario hacer este proceso otra vez. El problema es que el motor de *Unity* no elimina el objeto justo en ese instante, sino que lo hace al final de la iteración del bucle que se encarga de llamar a estas diferentes funciones. Por lo cual, la función *Start()* podría ser llamada antes de destruirse el *GameObject* y ser ejecutada.

En el resto de las líneas, hasta la 13, se declaran los *topics* en los que el script publica en el servidor público *HiveMQ* y a los que se suscribe. Las líneas 15 y 17 sirven para acceder al campo privado *messageHandler* del objeto *Dron* de la librería *csDronLink*. Este elemento pertenece a la clase *Dron* y contiene todos los métodos necesarios para establecer las diferentes conexiones. Al ser privado no se puede acceder mediante métodos directos de C#, por lo tanto, hay que seguir un proceso diferente, usando *GetField()* para obtener la variable *messagehandler* de la clase *Dron* de *csDronLink*, y posteriormente usando el método *GetValue()* para obtener el valor de *messagehandler* de un objeto concreto de la clase *Dron*.

En la línea 20 se llama al método *Task.Run()* para que encole la función *ConnectToBrokerAsync()* (ver figura 5.8) al *threadpool* y se le asigne un hilo separado del principal de *Unity*.

Extractos de la función ConnectToBrokerAsync() de DronMqttBridge.cs

```
await Task.Delay(TimeSpan.FromSeconds(5));  //1

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

await _client.ConnectAsync(options);  //2
```

Tabla 5.8: Extractos de la función *ConnectToBrokerAsync()* del script *DronMqttBridge.cs.*

ConnectToBrokerAsync() se encarga de crear el cliente MQTT y conectarse al servidor *HiveMQ*. Es un método de tipo *async*, lo cual significa que es asíncrono y su ejecución puede ser pausada mediante un *await*, como se puede observar en la tabla 5.8 en las líneas 1 y 2. Si durante la ejecución de un método async se lee un *await* junto a una tarea que aún no ha sido resuelta, la función queda pausada. Estas operaciones que no pueden ser procesadas al momento suelen ser de tipo I/O (*input/output*), como la de la línea 2 de la tabla 5.8, que son procesos que consisten en esperar datos en la red y que no tienen una duración fija.

Dentro del *threadpool* están los hilos llamados *completionPortThreads*, los cuales tienen la función de esperar un aviso del sistema operativo sobre si las operaciones I/O por las cuales están esperando han sido resueltas. Por otra parte, están los hilos genéricos *workerThreads*, dedicados a ejecutar el código del propio proyecto. Cuando estas tareas I/O son completadas, el SO manda una notificación al hilo *completionPortThread* encargado de controlar ese proceso. Cuando este recibe el aviso, el hilo encola en el *threadpool* la función que había sido pausada a la espera de la resolución de ese mismo proceso. Una vez en la cola, un *workerThread* retoma su ejecución. Las tareas junto a un *await* también pueden ser de tipo temporizador, como se puede observar en la línea 1 de la tabla 5.8. Estas tienen la característica que pausan la función un tiempo específico. Cuando transcurre este intervalo, se reanuda la ejecución de la función.

La función *ConnectToBrokerAsync()* se lanza con *Task.Run()*, y como consecuencia se ejecuta y se reanuda siempre en hilos del *threadpool*, nunca en el hilo principal de *Unity*, el cual queda libre para seguir generando los fotogramas del juego. Además, gracias a los *await*, mientras se espera una operación I/O no se ocupa ningún hilo, ya que el que estaba ejecutando la función recién pausada vuelve al *threadpool* y puede dedicarse a otras tareas en lugar de quedarse bloqueado esperando. Esto hace el código mucho más eficiente.

En las siguientes líneas de la función *Start()* (ver tabla 5.7), desde la 21 hasta la 24, se ejecutan diferentes corrutinas que ponen en marcha todas las comunicaciones de *Unity* con aplicaciones externas.

Otro objeto importante del Hierarchy es el *dron*. Este tiene asociado el script *DronControl.cs*. Se encarga de obtener información sobre las coordenadas geográficas y orientación del dron físico a través de la función *GetTelemetrySnapShot*() (ver línea 5 de la tabla 5.9) del script *DronMqttBridge.cs* (ver tabla 5.10). La llamada a este método se hace desde la función Update(), la cual es llamada automáticamente por el motor de *Unity* una vez por *frame*.

Cuando estos datos son obtenidos es necesario hacer una transformación de coordenadas esféricas a cartesianas para que cada posición del mundo real corresponda a un punto específico del escenario plano de *Unity*. Esta conversión es hecha dentro de la misma función *Update()* (ver tabla 5.9) en las líneas 8-11.

Función Update() de DronControl.cs

```
void Update()
{
        var bridge = ARDashboard.DroneMqttBridge.Instance;
        if (bridge == null) return;

        var snap = bridge.GetTelemetrySnapshot(); // 5

        if (!snap.hasData || (snap.lat == 0f && snap.lon == 0f)) return;


        float cosLat = Mathf.Cos(snap.lat * Mathf.Deg2Rad);  //8

        float x = snap.lon * cosLat * METERS_PER_DEG_LAT * sceneScale - 166339.2f;  //9

        float z = snap.lat           * METERS_PER_DEG_LAT * sceneScale - 4594877f;  //10

        float y = snap.alt * sceneScale;  //11


        Vector3    targetPos = new Vector3(x, y, z);

        Quaternion targetRot = Quaternion.Euler(-snap.pitch, snap.heading, -snap.roll);


        transform.position = Vector3.Lerp(transform.position, targetPos, Time.deltaTime *
        lerpSpeed);

transform.rotation = Quaternion.Slerp(transform.rotation, targetRot, Time.deltaTime * lerpSpeed);

}
```

Tabla 5.9: Función *Update()* del script *DronControl.cs.*

Función GetTelemetrySnapShot() de DronMqttBridge.cs

```
public TelemetrySnapshot GetTelemetrySnapshot()
{
        lock (_lock)
        {
                return new TelemetrySnapshot
                {
                        hasData = _hasData, lat = _lat, lon = _lon, alt = _alt,
                        heading = _heading, roll = _roll, pitch = _pitch,
                };
        }
}
```

Tabla 5.10: Función *GetTelemetrySnapShot()* del script *DronMqttBridge.cs.*

Volviendo al *Hierarchy* (ver figura 5.8), en el interior del objeto dron está el elemento *Camera*, que a la vez en su interior tiene el elemento *VideoQuad*. Que un *GameObject* esté en el interior de otro implica que, si el objeto principal se desplaza por el escenario, el elemento de dentro de él hará el mismo movimiento.

El objeto *Camera* de *Unity* es importante que tenga el mismo campo de visión que la cámara real del dron, porque, si una captura más imagen que la otra, los objetos virtuales no parecerá que estén realmente en el escenario real, ya que, al hacer giros o desplazamientos, el cambio de posición de estos en la pantalla no será igual al desplazamiento de los elementos que haya realmente en el mundo real. Debido a esto es importante introducir los parámetros adecuados en el *Inspector* del objeto *Camera* para que se asemeje lo máximo posible a la real (ver figura 5.9).

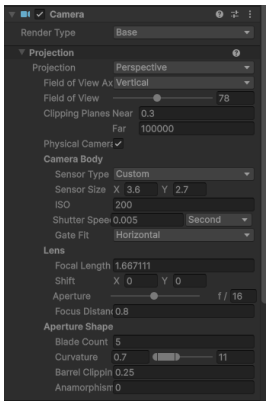


Figura 5.9: Parámetros del elemento *Camera.*

El *VideoQuad* es un elemento creado por el script *VideoBackground.cs*. Este código está adjunto al objeto *VideoBackground* que se puede observar en el *Hierarchy* y se encarga también de obtener el vídeo de la cámara conectada al ordenador. Este objeto va a actuar como pantalla para renderizar los datos provenientes de la fuente de vídeo. Ocupará todo el campo de visión del objeto *Camera*. Por lo tanto, en el fondo de la pantalla del juego en ejecución siempre se va a observar el vídeo del mundo real. El elemento al ser en el interior del objeto *Camera* y este a la vez dentro del objeto *dron*, todo el conjunto se moverá de la misma forma que el UAV. Es decir, si el dron hace un giro, el *VideoQuad* hará el mismo giro que este, teniendo como centro de rotación la posición del propio cuadricóptero, y si el UAV se mueve hacia cualquier dirección, este realizará el mismo desplazamiento.

La superposición del vídeo de fondo con los objetos virtuales del escenario de *Unity* produce el efecto de realidad aumentada buscado al ser plasmada en la pantalla del jugador (ver figura 5.10).

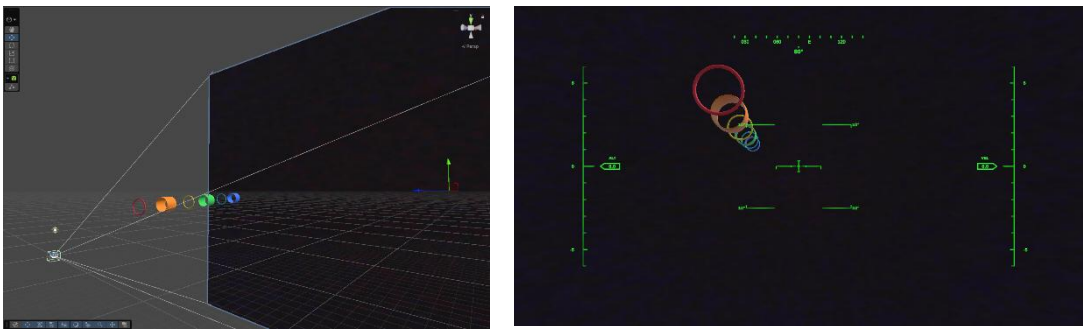


Figura 5.10: Comparación entre ventana *scene* y ventana *Game* cuando el juego está en ejecución.

En el *VideoQuad* se observa la imagen de la cámara gracias a que se ha creado un *Material* específico para este objeto. Contiene la información del vídeo en una variable llamada *textura* y tiene asociado un *shader* llamado *FishEyeUndistort*. Lo que hace este *shader* es extraer la información de los píxeles de la imagen de la cámara, los cuales vienen compactados en la variable *textura*, aplica la corrección correspondiente de ojo de pez y reubica la posición de cada píxel para que la imagen resultante tenga la rectificación aplicada.

El objeto *DronModel* tiene los componentes necesarios para darle apariencia al propio objeto *dron*.

El componente *FlightHud* que se observa tiene el script asociado *FlightHud.cs*. Este crea un *Canvas* transparente sobre el que se van a colocar todos los elementos del HUD.

Otro elemento del *Hierarchy* que aparece es el objeto *pause*. Este tiene asociado el script *Pausemenuui.cs*, el cual se encarga de construir un *Canvas* nuevo con el menú de pausa cuando se clica la tecla escape.

Los objetos llamados *Aros* son los elementos que verá el usuario en realidad aumentada dentro del escenario.

5.5.2 Disparar al blanco

A diferencia del resto de juegos, al clicar el botón “Iniciar” desde el menú de configuración con este modo seleccionado se abrirá otro menú de ajustes de juego (ver figura 5.11).

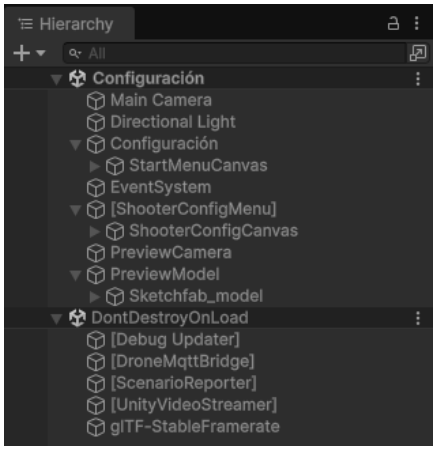


Figura 5.11: *Hierarchy* del juego *Disparar al blanco* cuando el juego está en ejecución.

Se observa que la escena no ha cambiado, sigue siendo la misma que la del menú de configuración. La diferencia es que ha aparecido un nuevo *Canvas* creado por el objeto *ShooterConfigMenu*, el cual tiene asociado el script *ShooterConfigMenu.cs*, que se encarga de construir los elementos de este. Otros objetos que se han generado son el *PreviewModel* (diseño 3D del enemigo) y el *PreviewCamera*. En vez de crear una escena separada para el previsualizado como se ha hecho en el editor de misiones, se ha colocado una cámara apuntando al modelo 3D en la misma escena (ver figura 5.12).

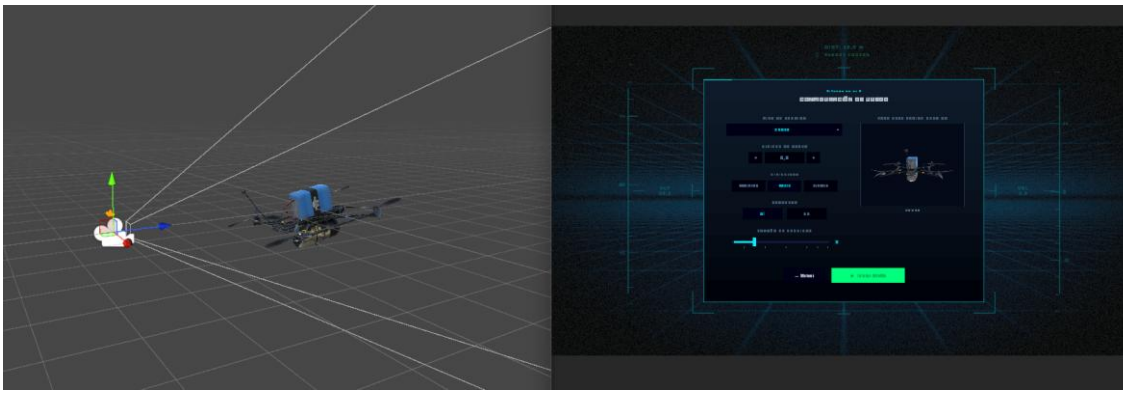


Figura 5.12: Comparación sobre cómo se observa el elemento *dron* en la ventana s*cene* y como se visualiza en el menú de configuración del juego.

En la figura 5.12 se observa cómo la cámara captura la imagen del modelo y como posteriormente aparece en el *Canvas*. Es importante que haya una cámara dedicada, ya que el modelo gira alrededor de su eje central. Por lo tanto, si solo se hiciera una captura de este no se conseguiría que se viese el movimiento giratorio en el menú.

Una vez se selecciona el botón “Iniciar misión”, se abre la escena del juego. Los elementos del *Hierarchy* de esta escena son los siguientes (ver figura5.13).

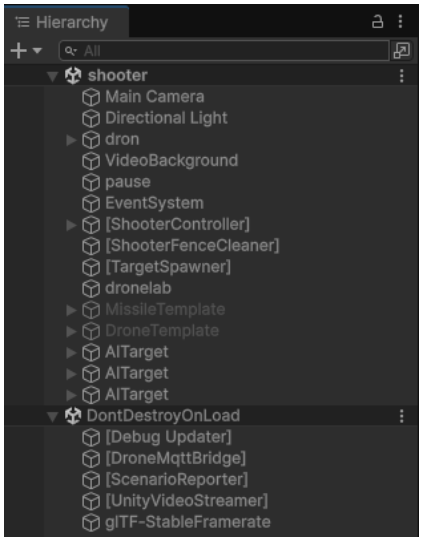


Figura 5.13: *Hierarchy* de la escena *shooter* correspondiente al juego *Disparar al blanco.*

La primera diferencia que se observa respecto a los juegos hechos desde el editor de misiones es que hay un objeto llamado *ShooterController*. Este se encarga de construir un *Canvas* en el que se van a colocar todos los elementos del HUD. Como este está en modo *screenspaceoverlay*, se dibuja encima de la pantalla del juego directamente. También es el responsable de detectar si el jugador ha hecho alguna acción con el teclado, el ratón o desde la *WebApp*, y, en caso afirmativo, ejecuta la función correspondiente a esa acción. Por ejemplo, si el jugador hace clic izquierdo con el ratón, este script ejecutaría la función *shoot().* Este método crea el objeto *projectile* en la escena con el script asociado *projectile.cs*. Este código se encarga del movimiento de este y también de detectar si ha impactado contra algún enemigo. En caso de que esté el arma *MISSILE* seleccionada en vez del *LASER*, se utiliza el script *missile.cs* para generar el proyectil.

En el *Hierarchy* (ver figura 5.13) también aparece el objeto *TargetSpawner* con el script asociado *TargetSpawner.cs*. Este código tiene la función de regenerar a los enemigos mediante la función *SpawnTarget()* a medida que el jugador los va eliminando. Para generarlos toma en consideración los límites del propio *dronlab*. También se encarga de enviar las *geofence* al piloto automático del dron real con la función *UploadGeofence(),* usando el script *DroneMqttBridge.cs* como intermediario.

Otros objetos que se pueden ver en el *Hierarchy* (ver figura 5.13) son el *DroneTemplate* y el *MissileTemplate*. Es necesario que estén ahí porque evita que el script *TargetSpawner.cs* y *Missile.cs* tengan que recrear el modelo 3D de estos objetos cada vez para generarlos. Estando ya en escena pueden ser copiados directamente.

El objeto *dronelab* es creado por *TargetSpawner.cs* mediante la función *CreateDroneLab().* Este sirve como referencia para saber la zona por donde pueden aparecer y moverse los enemigos.

El resto de objetos importantes son los *AITarget*. Estos son propiamente los enemigos generados por el *TargetSpawner.cs.* Tienen asociado el script *AITarget.cs*. Este tiene la función de controlar el movimiento del propio enemigo. El objeto *dronelab* también sirve como referencia para que las funciones encargadas del movimiento dentro de *AITarget.cs* sepan cuál es la zona por la que se pueden mover los oponentes. Cuando un enemigo es eliminado por un disparo del jugador, el script *projectile.cs* o *missile.cs* llama a la función *Hit()* de *AITarget.cs* para que el propio objeto enemigo reciba la información de que ha sido impactado. Dentro del método *Hit()* se llama a la función *NotifyHit()* del script *ShooterController.cs* para que se actualice el número de derribos en el HUD. También llama a la función *NotifyDestroyed()* para advertir a *TargetSpawner.cs* que un adversario ha sido eliminado. Finalmente, dentro de *NotifyDestroyed()* se llama a la otra función *SpawnTarget()* para que sea regenerado el enemigo.

El último componente del juego que falta por explicar es el encargado de los disparos de los enemigos. Cuando un adversario decide lanzar un proyectil, *AITarget.cs* llama a la función estática *Create()* de dentro del script *EnemyProjectile.cs*. Esta crea el objeto *EnemyProjectile* dentro del *Hierarchy*, el cual sale disparado hacia la posición que el dron del jugador ocupa en ese momento. En caso de que el proyectil detecte que ha impactado contra el jugador, el script *EnemyProjectile* llama a la función *TakeDamage()* de *ShooterController.cs*, el cual se encargará de restar el daño recibido en la barra de vida que tiene el jugador dentro del HUD. Posteriormente a este proceso, el disparo se autodestruye.

5.5.3 Laberintos

Hay tres laberintos distintos, pero todos tienen la misma estructura. Para esta explicación se va a explicar el *Laberinto2*. El *Hierarchy* es el siguiente (ver figura 5.14).

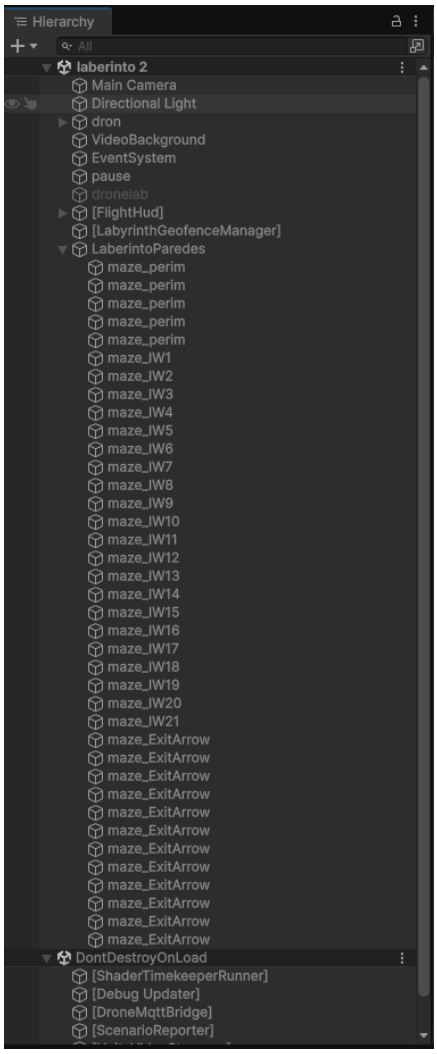


Figura 5.14: *Hierarchy* de la escena *Laberinto 2*.

Se puede observar que hay una escena dedicada llamada *laberinto 2*. El objeto más importante es el *LabyrinthGeofenceManager*. Este tiene asociado el script *LabyrinthGeofenceManager.cs* , el cual se encarga de construir y posicionar todas las piezas del laberinto, que se pueden observar en el interior del objeto *LaberintoParedes,* en el escenario de juego. También se encarga de enviar las *geofences* al autopiloto del dron real a través de la función *Start()*, la cual utiliza la instancia de la clase *DronMqttBridge* como intermediaria.

5.6 WebApp

Finalmente la última parte de la aplicación son las *WebApps*. Ambas tienen la misma estructura, solo que la dedicada a observadores incluye menos herramientas. Se va a explicar el funcionamiento de la aplicación web que incluye todas las funcionalidades.

En esta intervienen varios scripts. Por una parte están los ficheros que se ejecutan fuera del programa de *Unity* y por otra los que están integrados dentro de este. Respecto a los externos, uno de los más importantes es *app.py*. En este se inicializa la aplicación de *Flask*. Funciona como un servidor web, donde se registran todas las URL que el cliente puede visitar dentro de la página.

Rutas HTTP asociadas a funciones del script app.py

```
@app.route('/') //1
def index():  //2
    print(f"[INFO] Cliente conectado desde IP: {request.remote_addr}") //3
    return render_template('index.html')  //4

@app.route('/arm', methods=['POST'])  //5
def arm_route():  // 6
    data = request.get_json() or {}  //7
    publish_cmd('arm', {'force': bool(data.get('force', False))})  //8
    return jsonify({'status': 'ok'})  //9

@app.route('/takeoff', methods=['POST'])
def takeoff():
    data = request.get_json() or {}
    meters = data.get('meters', 5)
    publish_cmd('takeoff', {'meters': meters})
    return jsonify({'status': 'ok'})
```

Tabla 5.11: Ejemplo de diferentes rutas HTTP con funciones asociadas.

Si el usuario por ejemplo visita la ruta '/arm', HTTP enviará una petición *post* al servidor. La aplicación *Flask* leerá la URL de esta petición y ejecutará la función *arm_route(),* como se puede observar en la línea de código 6 de la tabla 5.11. En un primer momento, cuando el cliente entra por primera vez a la dirección de la página web, lo que recibe el servidor es una petición *get* con la URL de entrada (‘ / ’). Entonces automáticamente se activa la función *index()* del script *app.py*, como se puede visualizar en la línea 2 de la tabla 5.11, que envía el archivo *index.html* con toda la información sobre el formato y diseño de la web. Al final de este fichero se indica que el navegador debe enviar otra petición *get* al servidor para recibir el archivo *main.js*. Este archivo de *JavaScript* contiene toda la información sobre cómo debe de reaccionar la página web ante cualquier input del cliente, como puede ser un clic derecho con el ratón sobre un botón. Otro archivo que aparece referenciado en el *index.html* y que debe ser solicitado mediante un *get* al servidor es el *manifest.json*. Este archivo lo utilizan los navegadores como *Chrome* para saber cómo comportarse si el usuario elige la opción "Añadir a pantalla de inicio" la página web.

Además, dentro de *app.py* se inicializa un cliente MQTT, el cual publica y recibe instrucciones desde el servidor público *MQTT*. De esta forma, el programa de *Unity* que también está conectado a HiveMQ y el servidor web se pueden comunicar.

Creación del cliente MQTT en app.py

```
mqtt_client = mqtt.Client()
mqtt_client.on_connect = _on_mqtt_connect
mqtt_client.on_message = _on_mqtt_message
mqtt_client.connect_async(MQTT_BROKER, MQTT_PORT)
mqtt_client.loop_start()
```

Tabla 5.12: Creación del cliente MQTT para conexión con servidor *HiveMQ*.

Por ejemplo, si desde la *WebApp* se manda la instrucción de armar el dron, la aplicación de *Flask* recibirá esta petición y ejecutará la función asociada a esa acción para que esta publique la instrucción en el servidor MQTT. De esta forma, el script *DroneMqttBridge.cs* de *Unity,* que es el propietario de la conexión con el servidor *HiveMQ*, recibirá esas directrices y las enviará al piloto automático del dron real a través de su conexión con este.

Al ejecutar el script *app.py*, este, a parte de iniciar el servidor web, también se encarga de inicar el servidor *WebRTC*. Lo hace llamando a la función *run_in_background()* del script *webrtc_server.py*, como se puede observar en la línea de código 6 de la tabla 5.12.

Inicio del servidor WebRTC en app.py

```
if __name__ == '__main__':
    print(f"[INFO] Iniciando Drone Control Station (HTTP, sin TLS)")
    print(f"[INFO] Abre http://localhost:5000 en tu navegador")
    print(f"[INFO] Instrucciones/telemetría vía MQTT ({MQTT_BROKER}:{MQTT_PORT}); "
          f"el puerto SITL/COM lo abre Unity, no esta webapp.")

    webrtc_server.run_in_background(host='0.0.0.0', port=WEBRTC_PORT) //6

   socketio.run(app, host='0.0.0.0', port=5000, debug=True,
                use_reloader=False, allow_unsafe_werkzeug=True)
```

Tabla 5.12: Inicio del servidor *WebRTC* en la línea 257 desde el script app.py.

Este servidor sirve para que puedan conocerse el emisor y los receptores. Una vez ambas partes tienen información sobre el otro extremo de la comunicación, estos ya pueden empezar a enviarse paquetes de red directamente entre ellos sin intermediarios, lo cual acelera considerablemente el flujo de datos.

La función *run_in_background()* lanza un nuevo hilo en el mismo programa que ejecuta *app.py* en el que se va a iniciar el servidor. En este se inicia un *loop* en el que se van a poder ejecutar funciones asíncronas de la librería *asyncio* de *Python*. Estas serán útiles de cara a poder gestionar de forma eficiente las peticiones que lleguen al servidor. En caso de no utilizarlas, cada vez que se necesitase un dato que aún no está disponible se pausaría el hilo entero y no podría seguir realizando otras tareas durante el tiempo de espera. Este problema surge muy a menudo en servidores en los que para realizar ciertas operaciones se necesitan datos del usuario que deben de viajar por la red y que no estan disponibles de forma instantánea. Las funciones asíncronas ofrecen la posibilidad de ser pausadas en caso de contener alguna operación que no pueda ser ejecutada de forma inmediata.

La gestión de estos métodos la realiza el bucle en el que se agregan este tipo de funciones. Cada iteración del loop consta de dos fases. En un primer momento se revisa si el sistema operativo ha resuelto alguno de los procesos de red que estaban pendientes. En caso afirmativo, el bucle llamaría al objeto *Future* asociado a ese proceso concreto al que se estaba esperando y se le asignaría el valor correspondiente al resultado de esa acción. Los elementos *Future* forman parte de la librería *asyncio* y representan la resolución de una operación que todavía no ha terminado. En el momento en el que se le asigna un valor concreto a un objeto de este tipo mediante el método *set_result()*, se encolan en la lista *_ready* (recopilación de todas las tareas pendientes de ser ejecutadas en esa iteración) todos los *callbacks* (función que se registra para ser ejecutada cuando se produce un evento determinado) asociados a ese elemento *Future*. Estos *callbacks* son los que se encargan de reanudar la ejecución de las funciones que habían sido pausadas y que estaban esperando el resultado de estas operaciones de red.

No solo se utiliza este proceso para esperar a que lleguen datos desde dispositivos externos, sino que también se pueden pausar funciones por un tiempo determinado y posteriormente reanudar su ejecución siguiendo el mismo procedimiento. En un inicio el bucle consulta un temporizador para controlar este tipo de interrupciones también.

La segunda fase del bucle consiste en procesar todos los *callbacks* que haya en la lista *_ready* del *loop*. En este tramo se reanuda la ejecución de las funciones que estaban esperando datos que ya están disponibles. También se retoma el procesamiento de los métodos que hayan finalizado su periodo de pausa.

En caso de que en una iteración del bucle no haya *callbacks* en la lista *_ready* y se esté esperando por ejemplo a que lleguen datos por la red, el sistema operativo pausaría la ejecución del hilo entero en el que corre el *loop* hasta que estuviese disponible esa información.

Función run_in_background() de webrtc_server.py

```
def run_in_background(host: str = "0.0.0.0", port: int = 8108,
                      tls_host: str = None, tls_port: int = None,
                      certfile: str = None, keyfile: str = None) -> threading.Thread:
    def _runner():
        loop = asyncio.new_event_loop()  //5
        asyncio.set_event_loop(loop)
        try:
            loop.run_until_complete(   //8
                _start_websocket_server(host, port, tls_host, tls_port, certfile, keyfile))  //9
        finally:
            loop.close()

    thread = threading.Thread(target=_runner, daemon=True)
    thread.start()
    return thread
```

Tabla 5.13: Función *run_in_background()* del script *webrtc_server.py*.

En la línea de código 5 de la tabla 5.13 se observa cómo se inicia el bucle que va a correr en el hilo del servidor *WebRTC*. El *loop* solo se detendrá si en algún momento se completa la ejecución de la función *_start_websocket_server(),* como se puede observar en las líneas 8 y 9.

Función _start_websocket_server de webrtc_server.py

```
async def _start_websocket_server(host, port, tls_host=None, tls_port=None,
                                  certfile=None, keyfile=None):
    handler = _make_handler()

    await websockets.serve(handler, host, port, ping_interval=None)  //4
    print(f"Servidor WebSocket WebRTC (Unity) iniciado en ws://{host}:{port}")

    if tls_host and tls_port and certfile and keyfile:
        ssl_context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
        ssl_context.load_cert_chain(certfile, keyfile)
        await websockets.serve(handler, tls_host, tls_port, ping_interval=None, ssl=ssl_context)
        print(f"Servidor WebSocket WebRTC (navegadores, TLS) iniciado en
         wss://{tls_host}:{tls_port}")

    await asyncio.Future()  //12
```

Tabla 5.14: *Función _start_websocket_server* del script *webrtc_server.py*.

El método de la tabla 5.14 es el que realmente arranca el servidor, como se puede observar en la línea 4. En este se observa el primer *await.* Esta palabra se utiliza para ejecutar el objeto *awaitable* que siempre aparece junto a esta. En este caso se trata de la inicialización del servidor *WebRTC*. Al ser un proceso que puede conllevar operaciones de red es posible que en algún momento de la ejecución no se disponga inmediatamente de los datos necesarios para seguir con la apertura del servidor. Para evitar malgastar tiempo de CPU mientras se espera a la resolución de alguna de estas operaciones, se agrega la palabra *await* al objeto que inicializa el servidor. De esta forma, en caso de ser necesario, se pausará la ejecución de la función *_start_websocket_server()* y se le pedirá al SO que paralice todo el hilo hasta que estén disponibles los datos necesarios, ya que en este punto no hay más tareas en la lista *_ready* del bucle.

Uno de los argumentos que se le pasa a la función *websockets.serve()* para inicializar el servidor es el método *handler*, como se puede observar en la línea 4 de la tabla 5.14. Cuando se ejecuta el servidor por primera vez, este abre el puerto TCP 8108, crea un objeto de tipo *Future* al que se le asignará un valor cuando llegue un cliente solicitando conectarse. Cuando llega un usuario que quiere establecer una conexión se acepta su petición y se le asigna el valor de “aceptar conexión” al elemento *Future*. Este tiene asociado una serie de *callbacks* en el momento en el que se le asigna un valor, que desencadenarán en la creación de un nuevo objeto *Future* para la aceptación del siguiente usuario y en la ejecución de la función *handler()*, la cual se encargará de la comunicación con ese cliente en concreto. Para cada usuario conectado habrá un *handler()* diferente ejecutándose en el bucle. Este método nunca acaba de procesarse a menos que el cliente se desconecte, ya que para atender a todas las peticiones e interacciones con este es necesario que el *handler()* siempre esté activo, aunque su ejecución se vaya pausando debido a las diferentes operaciones de red que se dan en la comunicación.

Previamente se ha dicho que el bucle no terminará hasta que finalice la ejecución de la función *_start_websocket_server().* Al instanciarse en la línea 12 de 5.14 un objeto future al que nunca se le asignará un valor junto a un *await*, la función queda permanentemente pausada y nunca finaliza su procesamiento. Por lo tanto, el bucle del servidor itera de forma indefinida.

En la parte de *Unity* hay varios scripts que se encargan de interactuar con la propia *WebApp*. El principal es *UnityVideostreamer.cs*. Este está asociado a un objeto llamado *UnityVideoStreamer*, el cual vive en la escena permanente *DontDestroyOnLoad.* Por lo tanto está presente en cualquier pantalla del juego. Primeramente se encarga de conectarse al servidor *WebRTC*. Para hacerlo, primero hace una consulta a un archivo *txt* llamado *webrtc_server.txt*, el cual está presente en la misma carpeta en la que se ejecuta el programa de *Unity*. En este archivo aparecen la dirección IP y puerto TCP en el que está el servidor *WebRTC*, los cuales pueden ser modificados por el usuario en caso de que sea necesario. Una vez obtenidos estos datos, el script ya puede conectarse al servidor y registrarse como “emisor”, para de esta forma empezar el intercambio de información con cualquier cliente que se conecte a la *WebApp* y, una vez establecida la conexión, empezar a enviar los datos de vídeo al usuario directamente sin intermediarios.

Otro script que se ejecuta dentro de *Unity es ScenarioReporter.cs*. Este también está dentro de la escena permanente *DontDestroyOnLoad*, y sirve para enviar a la *WebApp* información sobre la escena específica en la que se encuentra en ese instante la aplicación de *Unity*. La *WebApp* necesita saber esa información para ocultar o mostrar los diferentes controles, los cuales varían según el juego que esté jugando el usuario. La información es enviada mediante la función *Report()* de *ScenarioReporter.cs*, que a la vez llama a la función *PublishScenario()* de *DronMqttBridge.cs*, la cual envía este dato mediante protocolo MQTT al servidor *HiveMQ*.

CAPÍTULO 6: PRUEBAS

En este apartado se ha detallado una guía que debe seguir cualquier persona que se descargue el proyecto de *Unity* con la finalidad de seguir con el desarrollo de la aplicación. Si el usuario supera todas las pruebas que se indican a continuación, será señal de que la aplicación funciona correctamente.

6.1 Pruebas de instalación

1. Comprobación del contenido de la carpeta AR.

-Dentro debe de aparecer la carpeta *Assets* con el siguiente contenido:

- Carpeta *Plugins* con las siguientes librerías: *csDronLink.dll*, *MQTTnet.dll*, *SharpDX.dll* y *SharpDX.DirectInput.dll*.
- Carpeta *Code* con un total de 28 *archivos.cs*.
- Carpeta *Scenes* con 8 escenas.
- Carpeta *Shaders* con 3 *shaders*.

2. Comprobación del contenido de la carpeta *WebApp*.

-Debe de aparecer el archivo *app.py*, *app_observador.py* y *webrtc_server.py*. También tiene que haber un archivo llamado *requirements.txt*, con todas las librerías de *Python* necesarias. Concretamente se necesitan las siguientes:

- *Flask==3.0.3*
- *flask-socketio==5.3.6*
- *python-socketio==5.11.3*
- *eventlet==0.35.2*
- *paho-mqtt==1.6.1*
- *websockets==13.1*

-Dentro de la carpeta *templates* deben de estar presentes los archivos *index.html* y *index2.html*.

-Tiene que haber una carpeta llamada *static*. Esta debe de contener el fichero *manifest.JSON* y la carpeta *js* con los archivos *main.js* y *main2.js*.

6.2 Pruebas en simulación

- Entrar al juego “Prueba 1” y configurarlo en modo simulación con la webcam del ordenador como fuente de vídeo.
- Al iniciar el juego verificar que se observe el vídeo de la cámara del ordenador.
- Despegar el dron en el SITL de *Mission Planner* desde dentro del *dronlab* y comprobar que esa acción también es realizada por el UAV del juego de *Unity*.
- Una vez el dron esté en el aire, acercarse a una pared del *dronlab* y comprobar que la *geofence* frena el UAV.
- Iniciar el escenario “Laberinto 1”. Verificar que las *geofences* frenan al dron antes de atravesar una pared del laberinto.
- Conectarse a la *WebApp* en modo simulación. Validar que se recibe el vídeo del juego de *Unity*. El dron simulado debe de estar en tierra.

- Desde la *WebApp* enviar la instrucción “Armar” al dron de *Mission Planner* desde el panel de herramientas. En caso de que se haya cumplido esta instrucción dentro del SITL, enviar petición de despegue a 5 metros.
- Si el dron ha despegado, cambiar el modo de vuelo a *Loiter* y comprobar que las indicaciones de los *joysticks* son realizadas por el dron del SITL.

6.3 Pruebas en producción

- Conectar antena de telemetría y vídeo al ordenador y verificar que la cámara del dron físico está orientada hacia delante y alineada con el plano del suelo.
- Enchufar batería al dron y comprobar que este se enciende correctamente.
- Entrar al juego “Prueba 1” y configurarlo en modo producción, eligiendo los puertos COM de la antena de telemetría y de vídeo.
- Una vez se inicie “Prueba 1”, verificar que el vídeo de la cámara se observa en el juego.
- Despegar el dron real y observar si el UAV virtual de la aplicación de *Unity* realiza la misma acción.
- Iniciar el escenario “Laberinto 1”. Verificar que las *geofences* frenan al dron antes de atravesar una pared del laberinto.
- Conectarse a la *WebApp* en modo producción, eligiendo el puerto COM en el que está conectada la antena de telemetría. Validar que se recibe correctamente el vídeo del juego de *Unity*. El dron debe de estar en tierra en este momento.
- Desde la *WebApp* enviar la instrucción “Armar” al dron físico desde el panel de herramientas. En caso de que se haya cumplido esta instrucción, enviar petición de despegue a 5 metros.
- Si el dron ha despegado, cambiar el modo de vuelo a *Loiter* y comprobar que las indicaciones de los *joysticks* son realizadas por el dron del SITL.

CAPÍTULO 7: INTEGRACIÓN AL DRON ENGINEERING ECOSYSTEM

Con tal de integrar este proyecto en el DEE y facilitar a los futuros estudiantes que quieran seguir con el desarrollo de la aplicación la comprensión de la estructura del programa y de sus diferentes componentes, se ha creado un repositorio de *GitHub* en el siguiente enlace:

https://github.com/adriapascuali-hash/Realidad_Aumentada_Unity

El repositorio incluye:

- Un *README.md* con una guía de instalación y una explicación sobre cómo está estructurado el proyecto
- Una carpeta con el proyecto de *Unity* y todos los recursos usados en este.
- El código fuente de la librería *csDronLink.dll*, por si se quieren consultar las diferentes funciones y clases de esta que han sido usadas en el desarrollo del proyecto.
- Un conjunto de vídeos donde se muestran diferentes escenarios de la aplicación en ejecución, se explican diferentes aspectos del funcionamiento de esta y se hace un repaso de los elementos más importantes del programa.
- También se ha subido un enlace a esta memoria.

CAPÍTULO 8: SOSTENIBILIDAD DEL PROYECTO

8.1 ¿Quién se beneficia del proyecto?

Este proyecto se ha diseñado con la finalidad de que sea una herramienta tanto lúdica como de trabajo. A menudo se hacen visitas guiadas en el Campus UPC del Baix Llobregat en las que se accede al *dronlab* y se realizan diferentes actividades con drones. Con todos los juegos y módulos que incluye esta aplicación, los visitantes podrán disfrutar de una experiencia de pilotaje en realidad aumentada con UAVs, en la que podrán realizar diferentes circuitos, escapar de laberintos o enfrentarse a diferentes enemigos.

No obstante, la aplicación también puede ser usada con fines prácticos. En el editor de misiones se pueden crear todo tipo de escenarios en cualquier localización del mundo. Esto puede servir de utilidad en diferentes sectores profesionales que trabajen con drones FPV y necesiten combinar elementos del mundo real con objetos virtuales para marcar una ruta de vuelo, señalar una posición geográfica, delimitar una zona específica o definir un circuito concreto.

8.2 Escalabilidad del proyecto

La aplicación es una plataforma de juegos en realidad aumentada con drones que sirve como punto de partida para futuras implementaciones. Estudiantes que dediquen su TFG al DEE y quieran seguir con el desarrollo de este programa pueden añadir todo tipo de juegos y funcionalidades nuevas. El editor de misiones se puede expandir introduciendo todo tipo de nuevos objetos y elementos que den pie a escenarios más variados y dinámicos. El proyecto también permite la implementación de nuevos módulos, como puede ser una aplicación de móvil o una *WebApp* con funcionalidades nuevas.

8.3 Coste del proyecto

A nivel económico el costo monetario del proyecto ha sido muy bajo. El dron y el resto del *Hardware* han sido facilitados por la propia universidad. El desarrollo del *software* no ha supuesto ningún gasto, ya que las licencias de *Unity* y del resto de programas utilizados son gratuitas. El ordenador usado para desarrollar el proyecto ha sido el mío personal.

Por lo que respecta a las horas de trabajo invertidas, para desarrollar todo el proyecto he tenido que dedicar un total de 450 horas aproximadamente y 20 horas de reuniones con el tutor.

CAPÍTULO 9: CONCLUSIONES

9.1 Objetivos alcanzados

El proyecto ha cumplido con todos los objetivos iniciales propuestos en la planificación. Se ha conseguido crear una plataforma de juegos en realidad aumentada con diversos videojuegos y un menú de edición de misiones, el cual permite al usuario crear todo tipo de escenarios en cualquier sitio del mundo con una gran variedad de objetos y elementos.

La interfaz gráfica se ha diseñado para que sea lo más intuitiva y agradable posible a nivel visual para el usuario, con una combinación de colores y elementos acordes al tipo de aplicación creada. El juego ofrece la posibilidad de jugar en modo simulación con el SITL de *Mission Planner* o en modo producción con el dron real, lo cual permite probar escenarios y juegos desde casa. La jugabilidad es agradable y los objetos virtuales se observan de forma suficientemente realista en realidad aumentada. El vídeo proveniente de la cámara del dron llega a *Unity* con muy poca latencia y con mucha fluidez, lo cual da al usuario una buena inmersión en el juego.

El juego de “Disparar al blanco” tiene un sistema de dificultad progresivo que reta al jugador a mejorar sus habilidades y los enemigos tienen un sistema de movimiento pulido y trabajado.

Además, se ha desarrollado una *WebApp* para jugadores que permite controlar el dron desde un dispositivo con acceso a internet. Esta tiene un diseño cuidado con una buena distribución de los elementos para que resulte lo más confortable posible para el usuario. Presenta dos *joysticks* y un panel de herramientas desde donde se pueden enviar diferentes instrucciones al dron. En esta también se puede observar el vídeo del juego de *Unity* con una gran fluidez, gracias a la tecnología *WebRTC*. Para observadores que solo deseen visualizar la pantalla del juego sin elementos de control que interfieran, se ha creado otra *WebApp* con este fin a la que se podrán conectar.

9.2 Líneas de desarrollo abiertas

La aplicación puede mejorar en múltiples aspectos. Uno de los cambios que mejoraría considerablemente la experiencia del jugador sería conseguir que los datos de posición del dron real se actualizasen a un ritmo más alto. Esto aumentaría la inmersión y mejoraría la experiencia de juego, ya que el efecto de realidad aumentada sería aún más realista. En cuanto a los juegos, se podrían hacer nuevas propuestas que dotasen a la plataforma de una gran variedad de posibilidades de entretenimiento. En el editor de juegos, se podrían añadir más objetos y elementos que permitiesen al usuario crear escenarios todavía más variados. También sería útil que la *WebApp* permitiera al usuario interactuar con los diferentes menús de la aplicación de *Unity* directamente desde su dispositivo móvil o tableta, y de esta forma no necesitar manipular el ordenador para estas tareas.

Otra posible mejora sería utilizar un bróker privado en la red en lugar de uno público como *HiveMQ*, al cual todo el mundo tiene acceso. Algún individuo con malas intenciones que conociese los *topics* en los que la *WebApp* pública instrucciones en el servidor MQTT podría mandar indicaciones maliciosas al dron. También sería una mejora de seguridad usar el protocolo *https* en vez de *http* en el servidor web, para que de esta forma la información viaje cifrada por la red y evitar que un tercero pueda alterar o visualizar el contenido de un paquete de la conexión. Este cambio junto a un buen sistema de autenticación para acceder a la *WebApp* dificultaría mucho a un atacante tomar el control del dron.

9.3 Valoración personal

Desarrollar este proyecto ha aumentado considerablemente mis conocimientos sobre los drones y la programación aplicada a estos. Antes de empezar este trabajo, mis nociones sobre redes o *Unity* eran bastante limitados. Gracias a realizar este proyecto he entendido mucho mejor cómo los UAVs se comunican con los diferentes dispositivos en tierra, qué protocolos usan y cómo gestionan el intercambio de información. También he aprendido mucho sobre el funcionamiento de *Unity*. Es un programa que permite hacer todo tipo de proyectos 3D, y ofrece una libertad absoluta a cualquier idea que se quiera desarrollar. Gracias a haber hecho la aplicación de realidad aumentada y la *WebApp*, tengo una visión mucho más amplia sobre todas las posibilidades que ofrecen los drones. Pienso que ha sido una buena decisión dedicar mi TFG a este proyecto. Con el conocimiento adquirido durante el desarrollo de este trabajo me siento mucho más preparado y capacitado para afrontar los retos a los que me pueda enfrentar en un futuro.

REFERENCIAS

[1] dronsEETAC, "Drone Engineering Ecosystem DEE," GitHub. [En línea]. [Consulta: 05 Septiembre 2026]. Disponible en: https://github.com/dronsEETAC/DroneEngineeringEcosystemDEE

[2] dronsEETAC, "Librería dronLink para C#," GitHub. [En línea]. [Consulta: 05 Septiembre 2026]. Disponible en: https://github.com/dronsEETAC/csDroneLink

[3] dronsEETAC, "Tutorial de video streaming para el Drone Engineering Ecosystem," GitHub. [En línea]. [Consulta: 08 Septiembre 2026]. Disponible en: https://github.com/dronsEETAC/Tutorial_VideoStreaming

[4] Unity Technologies, "Manual de Unity". [En línea]. [Consulta: 09 Septiembre 2026]. Disponible en: https://docs.unity3d.com/es/530/Manual/UnityManual.html

[5] Unity Technologies, "glTFast | 3D Formats | 6.18.0". [En línea]. [Consulta: 07 Septiembre 2026]. Disponible en: https://docs.unity3d.com/Packages/com.unity.cloud.gltfast@6.18/manual/index.html

[6] Unity Technologies, "WebRTC | 3.0.0-pre.8". [En línea]. [Consulta: 03 Septiembre 2026]. Disponible en: https://docs.unity3d.com/Packages/com.unity.webrtc@3.0/manual/index.html

[7] Unity Technologies, "Universal Render Pipeline | 17.3.0". [En línea]. [Consulta: 04 Septiembre 2026]. Disponible en: https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.3/manual/index.html

[8] mob-sakai, "UIEffect," GitHub. [En línea]. [Consulta: 01 Septiembre 2026]. Disponible en: https://github.com/mob-sakai/UIEffect

[9] MQTTnet, "MQTTnet," GitHub. [En línea]. [Consulta: 02 Septiembre 2026]. Disponible en: https://github.com/dotnet/MQTTnet

[10] Newtonsoft, "Json.NET". [En línea]. [Consulta: 05 Septiembre 2026]. Disponible en: https://www.newtonsoft.com/json

[11] OpenCV, "Camera Calibration and 3D Reconstruction". [En línea]. [Consulta: 08 Septiembre 2026]. Disponible en: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

[12] OpenCV, "Camera Calibration" (tutorial). [En línea]. [Consulta: 07 Septiembre 2026]. Disponible en: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

[13] bohmerang, "Missile & Bomb Collection - Fighter Jets - Free," Sketchfab. Licencia CC-BY-NC-SA-4.0. [En línea]. [Consulta: 03 Septiembre 2026]. Disponible en: https://sketchfab.com/3d-models/missile-bomb-collection-fighter-jets-free-d203b678424a43568703bf85ba680f47

[14] Google Fonts, "Orbitron". [En línea]. [Consulta: 01 Septiembre 2026]. Disponible en: https://fonts.google.com/specimen/Orbitron

[15] Google Fonts, "Share Tech Mono". [En línea]. [Consulta: 06 Septiembre 2026]. Disponible en: https://fonts.google.com/specimen/Share+Tech+Mono

[16] Google Fonts, "Noto Sans Symbols". [En línea]. [Consulta: 05 Septiembre 2026]. Disponible en: https://fonts.google.com/noto/specimen/Noto+Sans+Symbols

[17] Google Fonts, "Iceland". [En línea]. [Consulta: 12 Septiembre 2026]. Disponible en: https://fonts.google.com/specimen/Iceland

[18] Microsoft, "Documentación de la familia de productos de Visual Studio," Microsoft Learn. [En línea]. [Consulta: 15 Septiembre 2026]. Disponible en: https://learn.microsoft.com/es-es/visualstudio/?view=visualstudio

[19] Claude Sonnet 5, Anthropic. "Asistencia en el desarrollo del proyecto AR". [en linea]. [Consulta: 04 Septiembre 2026]. Disponible en: https://claude.ai/